Braid: Sybil-Resistant Decentralized Identity with Trustless Key Recovery and Non-Transferable Anonymous Credentials

Anonymous Authors

Abstract

Decentralized identity frameworks grant users full sovereignty over their digital assets in the Web3 ecosystem. However, allowing arbitrary creation of identifiers makes the system susceptible to Sybil attacks and puts assets at risk when keys are lost or compromised. Moreover, the lack of identification prevents anonymous credential schemes from deterring malicious transfers. While existing solutions attempt to address these issues by linking identifiers to entities through trusted intermediaries, these entities are not always accessible and require costly offline interactions.

In this work, we introduce Braid, a decentralized identity system offering Sybil resistance, trustless key recovery, and non-transferable anonymous credentials. Braid creates blockchain-based bindings between identifiers and gradually combines identifiers belonging to the same holder into a unified associated identifier. As all identifiers within an association are presumed to belong to one individual, any fraudulent activity can be detected. The association grows larger as interactions increase, substantially reducing the likelihood of successful Sybil attacks. This mechanism allows holders to recover identifiers with lost keys or partial key compromise by proving knowledge of specific association structures. Additionally, Braid prevents unauthorized transfers through blockchain-based identifier-key bindings and proofs of ownership for credentials. The evaluation shows that Braid effectively bridges the gap between privacy-preserving credentials and accountable identity, achieving identifier association and credential presentation times of 2.41s and 3.31s on consumer-grade devices.

CCS Concepts

• **Security and privacy → Distributed systems security**.

Keywords

Decentralized identity, anonymous credentials, Sybil resistance, trustless key recovery, zero-knowledge proofs

ACM Reference Format:

Anonymous Authors. 2026. Braid: Sybil-Resistant Decentralized Identity with Trustless Key Recovery and Non-Transferable Anonymous Credentials. In *CCS '26: Proceedings of the 2026 ACM SIGSAC Conference on Computer and Communications Security, November 15-19, 2026, Netherlands.* ACM, New York, NY, USA, 21 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Cyberattacks on centralized identity systems have led to serious data breaches, resulting in financial losses and increased costs for enterprises and governments. Decentralized identity, integral to Web3, leverages decentralized ledgers and verifiable credentials to promise users unprecedented sovereignty and privacy [57, 58]. This vision enables participation in on-chain activities like decentralized autonomous organization (DAO) voting or token airdrops with minimal data disclosure, seemingly eliminating the risks of identity tracking and data breaches common in centralized systems [64].

However, this promising vision is fundamentally undermined by three persistent and interconnected challenges that severely limit its practical application. The first challenge is the foundational threat of the *Sybil attack*. The very feature that enhances privacy—the ability to create multiple identifiers—creates a fertile ground for malicious actors. A single attacker can masquerade as thousands, illegitimately draining airdrops intended for a community or seizing control of a DAO's treasury, thus corrupting the core principle of fair, decentralized governance [28, 45].

The second challenge comes from the brittle nature of *self-sovereign key recovery*. Compared to centralized systems, user-created keypairs are the sole gateway to digital assets. This absolute control becomes an absolute vulnerability upon key loss. The consequences are dire: an estimated $140 billion in digital assets lie stranded in inaccessible wallets, a permanent testament to the unforgiving nature of this challenge [1].

The third challenge arises from the inherent contradiction between *anonymity* and *non-transferability*. Anonymous credentials are vital for privacy, shielding users from collusion and surveillance [24, 53]. However, this anonymity makes them transferable, creating a bustling black market for stolen credentials, evidenced by the 80 million sold on the Genesis marketplace alone [56, 59]. Enforcing non-transferability has traditionally required sacrificing the very anonymity that makes these credentials powerful, forcing developers into a security paradigm with no good choices.

1.1 Prior solutions and limitations

Several solutions have been developed to tackle the challenges mentioned above while exhibiting certain limitations.

Sybil-resistant identification. Sybil attacks in decentralized systems cannot be eliminated entirely—they can only be deterred by making attack costs exceed potential gains [25]. In Web3 systems, offline gatherings serve as a prominent anti-Sybil strategy. Techniques such as Proof-of-Personhood (PoP) [8] and Worldcoin's iris scanning approach [31] require physical presence for identification. However, this approach faces major scalability challenges, increases ordinary user costs significantly, and remains vulnerable to low-cost attacks[1]. Collateralization, another traditional anti-Sybil method, creates high entry barriers while remaining susceptible to circumvention through token lending or user collusion—a vulnerability particularly evident in quadratic voting [4, 10, 29, 50].

[1] An attack known as *Puppeteer Attack* occurs when an attacker pays someone, typically for as little as $10, to undergo identification on its behalf [52].

CCS '26, Netherlands

One emerging strategy involves migrating legacy profiles [39] or natural unique identifiers [43], such as the U.S. Social Security Number (SSN). However, the inherent scope limitations of legacy systems restrict this approach's ability to serve all potential users. A related approach evaluates existing credentials to determine identifier trustworthiness [32]. While improving usability, this approach requires reassessment during each verification process to maintain privacy protection. We envision a progressive anti-Sybil solution that can both accumulate and carry forward the identifier trustworthiness demonstrated during each verification.

Key recovery. Social recovery has emerged as a prevalent remedial approach, enabling users to authorize a designated group of guardians to facilitate key recovery or modification in the event of key loss [35, 42, 65]. However, this strategy encounters challenges pertaining to the necessity of trust in the guardians and the requirement for offline verifications to authenticate the user's identity. Keys can be recovered without the owners' consent if these guardians collude [18, 36].

Credential verification with identity privacy. The challenge of providing non-transferable yet anonymous credentials remains unsolved, forcing existing approaches to trade off between these features [37]. Some methods sacrifice anonymity by requiring credential holders to reveal their identity during presentation [57, 65]. Other approaches, focusing on anonymous credentials, abandon non-transferability and instead limit the number of presentations to prevent unauthorized use [12, 51, 55].

1.2 Our contributions

In this paper, we introduce a comprehensive decentralized identity system entitled "*Binding and Aggregating Associated Identifiers* (Braid)." This system is designed to enable *progressive Sybil resistance* and *trustless key recovery*, while maintaining *non-transferability* and *anonymity* of verifiable credentials.

Braid establishes bindings between identifiers on the blockchain and progressively aggregates those belonging to the same holder into an associated identifier. Essentially, it strikes a balance between identity privacy and accountability. For legitimate holders, Braid records their associated identifiers on the blockchain without exposing the private information of individual identifiers. It also offers a way for these users to prove ownership of individual identifiers through the associated identifier. Conversely, for malicious holders, Braid stops malicious users from unfairly benefiting by forging multiple identities or transferring anonymous credentials, thanks to its binding and aggregating processes. In this way, Braid effectively addresses the limitations of previous approaches and resolves the three challenges above.

Progressive Sybil resistance. Braid leverages a pivotal observation that holders often need to present multiple credentials to participate in a campaign [65]. For instance, an airdropper might require holders to provide at least three participation credentials before qualifying for a new airdrop [28]. Braid achieves Sybil resistance through progressive binding of correlated identifiers. When users present multiple credentials at once, Braid enforces the aggregation of corresponding identifiers into an indivisible association, which is submitted to the blockchain in a cumulative manner. Given that all identifiers within an association are assumed to belong to the same individual, any attempt at duplicate participation within that association can be effectively detected. Theoretical analysis and experiments demonstrate that, as interactions increase, associations will expand progressively while the number of manipulable credentials remains limited, thereby substantially reducing the chance of successful Sybil attacks. Notably, Braid is carefully designed to handle cases where malicious holders try to deliberately spread a single identifier across multiple associations. This persistent cross-context linkage necessitates a strategic trade-off: Braid relaxes the strict global unlinkability of traditional anonymous credentials into *contextual unlinkability*. By prioritizing cross-context Sybil resistance over absolute isolation when public linkability handles are reused, Braid provides a practical security design for Web3 applications where Sybil resistance is critical, such as DAO governance and ecosystem-wide airdrops.

Trustless key recovery. Traditional social recovery mechanisms depend on trust relationships, using secret sharing between custodians to reconstruct keys after loss. In contrast, Braid provides *trustless* key recovery through its identifier association mechanism. Braid implements two approaches for reliably identifying genuine identifier owners. Holders can demonstrate ownership of any identifier in an association by proving they have the private keys of other identifiers within that association. Alternatively, when associated identifiers provide sufficient information entropy to meet security requirements, holders can prove ownership by demonstrating their knowledge of the hidden composition of the association without needing to memorize specific keys.

Non-transferable anonymous credential. The success of Braid hinges on the authenticity of credentials. In scenarios where identity privacy is crucial, credentials are often anonymous and vulnerable to transfer by malicious holders. To tackle this issue, Braid requires holders to bind their identifiers and private keys on the blockchain. When presenting anonymous credentials, holders must prove to verifiers that the credentials' holder identifiers align with those bound on the blockchain. As only the genuine holder possesses the private key and the proof is zero-knowledge, Braid can assert the credential owner without compromising identity privacy, preventing the transfer of anonymous credentials. Notably, these benefits do not require any modifications to existing credentials.

Extensions. Beyond the contributions mentioned above, Braid implements credential revocation and identifier blocklisting to enforce accountability for malicious behaviors. Braid extends tracking and penalties beyond just the offending identifier to include all associated identifiers controlled by malicious users. As a bonus feature, Braid empowers credential holders to designate specific verifiers, ensuring proofs work only for designated verifiers, thereby preventing malicious proof forwarding and impersonation attacks.

Contributions. To summarize, we design, implement, and evaluate Braid, which offers significant enhancements over pure anonymous credentials and existing decentralized identity solutions. Our key contributions include:

- A practical anti-Sybil mechanism named *identifier association* significantly raises Sybil attack costs while maintaining low user overhead and eliminating the need for external identity profiles, offline gatherings, or collateral.

- A *trustless key recovery* mechanism that enables holders to regain control over specific identifiers with lost keys without relying on trusted parties or authorities.
- *Non-transferable anonymous credentials* that authenticate credential ownership while protecting identity privacy.
- *Identifier blocklisting* and *credential revocation* for enhanced accountability, along with *non-forwardable proofs* to prevent replay attacks of designated verifier proofs.
- Security analysis and functionality evaluation, demonstrating Braid's progressive Sybil resistance and better performance compared to baseline identity and credential schemes.

2 Preliminaries

Notation. We denote by $\lambda \in \mathbb{N}$ the security parameter and by 1^λ its unary representation. Unless otherwise stated, we assume that all cryptographic algorithms in this paper take 1^λ as an input and thus omit it. If S is a finite set, we denote by $x \leftarrow S$ the process of sampling x according to S and by $x \leftarrow_\$ S$ a uniform one. We denote by $\mathsf{negl}(\lambda)$ a *negligible* function with respect to λ. We denote by $[\ell]$ the non-negative integer series up to ℓ, e.g., $\{1, 2, \ldots, \ell\}$. We say an algorithm is *probabilistic polynomial time (p.p.t.)* if it is probabilistic and operates within polynomial time with respect to λ.

In this paper, we use two collision-resistant hash functions H_a, H_n which can be modeled as random oracles. These hash functions can be constructed from a single primitive using proper domain separation. We also employ a key derivation algorithm PKGen whose one-wayness ensures that the public key does not divulge any knowledge about the private key.

Non-interactive zero knowledge. Let $\mathcal{RG}$ be a relation generator that takes a security parameter λ and returns a binary relation R. For a pair (x, w) such that $R(x, w) = 1$, we call x the *statement* and w the *witness*. Given 1^λ, we define the set of all relations that $\mathcal{RG}$ may output as $\mathcal{RG}_\lambda$. Using the current standard notation [19], we denote a proof for relation R as:

$$\pi = \mathsf{ZK\text{-}PoK}\{w : R(x, w) = 1\}.$$

For clarity, we will present only the relations $R(x, w)$ in protocols of Section 4, with the witness element(s) w highlighted to enhance understanding.

Cryptographic commitment. A commitment scheme is a pair of polynomial-time algorithms $(\mathsf{Commit}, \mathsf{Open})$ which performs as:

- $(c, r) \leftarrow_\$ \mathsf{Commit}(x)$: takes as input a message x, and outputs its commitment c and a randomness r.
- $b \leftarrow \mathsf{Open}(x, c, r)$: takes as inputs a message x, a commitment c and a randomness r, and outputs $b = 1$ for a valid opening or $b = 0$ otherwise.

Incremental Merkle tree. Merkle tree is a vector commitment scheme that commits to a vector $\boldsymbol{m} = (m_1, \ldots, m_\ell)$ and opens it at arbitrary index $i \in [\ell]$. An incremental Merkle tree is a constant-depth tree with placeholder leaves and pre-computed intermediate nodes. Elements are inserted sequentially by replacing the placeholders. An incremental Merkle tree could be abstracted as a tuple of polynomial-time algorithms $(\mathsf{Insert}, \mathsf{Auth})$ which performs as:

- $(\boldsymbol{m}', r, \rho) \leftarrow \mathsf{Insert}(x, \boldsymbol{m})$: takes as inputs a new leaf x and $\boldsymbol{m}$, returning a new vector $\boldsymbol{m}' = (m_1, \ldots, m_\ell, x)$, the updated root r and a inclusion proof ρ for x.
- $b \leftarrow \mathsf{Auth}(r, x, \rho)$: takes as inputs a leaf x, a Merkle root r and a proof ρ, returning $b = 1$ for a valid tuple (r, x, ρ).

Universally composable security. In this work, we employ the Universal Composability (UC) framework for security definition and analysis [20]. This framework involves a protocol Π interacting with an *adversary* $\mathcal{A}$ and an *environment* $\mathcal{E}$ with initial input z. $\mathcal{A}$ can send backdoor messages to parties, transmit outputs to $\mathcal{E}$, read outgoing messages from parties, and deliver messages between parties. When $\mathcal{A}$ corrupts a party, it gains full access to its messages and internal states.

The security of Π is defined by comparing its execution in the *real world* to an *ideal process*. The key ingredient in the ideal process is the *ideal functionality* $\mathcal{F}$, which captures the desired specifications and security properties of the task at hand. $\mathcal{F}$ is a Turing machine that interacts with $\mathcal{E}$ and an ideal adversary (*simulator*) $\mathcal{S}$ through a set of *dummy parties*. When $\mathcal{S}$ is activated, it can send messages to $\mathcal{F}$. $\mathcal{S}$ is also responsible for delivering messages from $\mathcal{F}$ to parties.

We say a protocol Π *UC-realizes* the functionality $\mathcal{F}$ if for any adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that no environment $\mathcal{E}$ can distinguish with non-negligible probability whether it is interacting with $\mathcal{A}$ and Π or with $\mathcal{S}$ and $\mathcal{F}$.

3 Overview

In this section, we start by introducing the system and threat models of Braid. Subsequently, we introduce the foundational concepts behind Braid, followed by its security and privacy objectives.

3.1 System and threat models

System model. Braid considers a decentralized identity system composed of *issuers* $\mathcal{I}$, *holders* $\mathcal{H}$, and *verifiers* $\mathcal{V}$. To participate in Braid, *holders* create a key pair $(\mathsf{pk}, \mathsf{sk})$ and publish the public key pk to a Braid-specific *verifiable data registry (VDR)* $\mathcal{R}$ maintained by a decentralized ledger (e.g., a blockchain) for registration. The VDR then generates a DID document that encompasses pk and an identifier[2] id that resolves to it. Crucially, *issuers* are not required to formally register within the Braid system[3]; they simply publish their public keys to any standard W3C-compliant registry. This allows verifiers to natively authenticate credentials without imposing adoption barriers on legacy issuers.

A *verifiable credential* $\mathsf{cred} = (\mathsf{id}^{\mathcal{I}}, \mathsf{id}^{\mathcal{H}}, \sigma, s)$ contains a collection of claims s issued by some $\mathcal{I}$ to certain $\mathcal{H}$, who can then present it to some $\mathcal{V}$ for authentication, as shown in Figure 1. In accordance with the W3C specification [57], cred contains identifiers $\mathsf{id}^{\mathcal{I}}$ and $\mathsf{id}^{\mathcal{H}}$ of its issuer $\mathcal{I}$ and holder $\mathcal{H}$, as well as a signature σ generated by signing other portions of cred using $\mathcal{I}$'s private key $\mathsf{sk}^{\mathcal{I}}$. A *campaign* is an interaction event (e.g., token airdrop) initiated by a verifier and identified by a *session ID* sid. Each participant can take part *only once* in a campaign.

Threat model. Our protocol considers malicious *holders* trying to bypass verification without possessing valid credentials or sharing credentials with other parties. Holders are allowed to register and

[2] All *identifiers* in this paper are decentralized identifiers (DIDs) that conform to the W3C specification [58], unless otherwise indicated.

[3] For clarity, we record issuer identifiers and keys in Braid's VDR in this manuscript. In practice, they may be maintained in any external registry accessible to all parties.

control multiple identifiers. Based on rational assumptions, holders will not share private keys with others.

Issuers, while executing the protocol honestly, maintain records of all issued credentials and may collude with verifiers to track holder activities and deduce their identities. Issuers will not collude with holders to issue fraudulent credentials. *Verifiers* are completely untrusted, as they may collude with issuers to obtain credential claims or holder identities, and may even forward holder proofs to impersonate their identities. However, we assume verifiers are economically motivated to prevent Sybil attacks to ensure the fairness and integrity of their own campaigns.

The *VDR* operates through blockchain smart contracts, with security based on standard permissionless blockchain assumptions where all states remain public and immutable. The VDR provides consistent observations to all observers at any moment. We also assume that network communications between parties remain protected from third parties and that all states except for the VDR are secure and private.

Behavioral assumptions. The security of Braid's cross-context Sybil resistance and non-transferability relies on rational behavioral assumptions driven by economic incentives in high-stakes environments. Specifically, we assume:

- *Credential scarcity:* Acquiring valuable, issuer-verified credentials requires significant capital, effort, or sustained contributions, whereas creating blank identifiers is costless.
- *Joint presentation:* Since individual credentials rarely match complex application requirements, verifiers naturally require the joint presentation of multiple credentials, forcing attackers to acquire distinct credential sets for each Sybil identity.
- *Incentive to reuse associations:* Credentials are bound to specific identifiers at issuance. Once associated, these identifiers are cryptographically locked. Creating new identifiers yields only credential-free blank slates. Thus, users must reuse and expand their existing associations.
- *Incentive against key sharing:* As in any public-key infrastructure, digital asset security relies on private key secrecy. Rational users will not share their keys, as doing so equates to relinquishing control over their assets.

3.2 Design concepts

To illustrate the core concepts of Braid, we present a running example featuring Alice, an active participant in the Web3 ecosystem. Alice wants to join *FutureDAO*, a high-value DAO. Through this example, we will show how Braid provides three key benefits: *Sybil resistance, anonymous but non-transferable presentation*, and *trustless key recovery*.

3.2.1 The challenge: Sybil attacks in high-stakes DAOs. *FutureDAO* governs a substantial treasury, making its voting process vulnerable to Sybil attacks. A simple *one-address-one-vote* model proves inadequate since attackers could create thousands of addresses at a negligible cost. To ensure fair governance, *FutureDAO* establishes a more robust policy: a member must prove it is a multifaceted contributor by presenting at least two distinct, valuable credentials. For instance, they might require a *Developer Credential* issued after verification of significant code contributions and a *Community Credential* awarded for sustained, active participation.

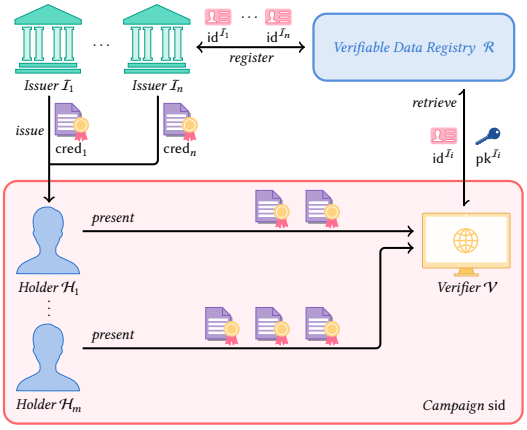

Figure 1: The flow of credential issuance and presentation.

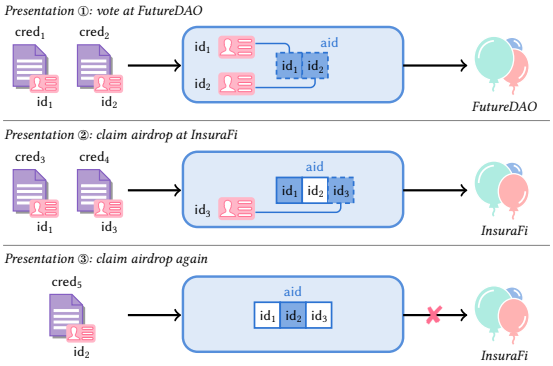

Figure 2: An example of the identifier association process.

This policy's effectiveness stems from two synergistic principles. *Credential scarcity* dictates that valuable credentials from reputable third parties are hard to acquire, unlike cheap identifiers [32, 39]. Simultaneously, the trend of *joint presentation* requires users to prove multifaceted qualifications in high-stakes applications [40, 46, 57]. This creates a prohibitively expensive dual challenge for attackers, who must acquire not just single credentials, but distinct pairs for each Sybil identity.

However, this scenario highlights limitations of traditional privacy-preserving schemes. Even an attacker who legitimately obtained one credential pair could use these credentials separately across different campaigns to build reputation, violating the principle of unified identity. This specific vulnerability is what Braid is engineered to address.

3.2.2 Braid's approach: enforcing identity association. As shown in Figure 2, Alice holds a *Developer Credential* cred_1 associated with the identifier id_1 and a *Community Credential* cred_2 associated with id_2. To vote in *FutureDAO*, she must present both. This is where Braid comes into play.

Identifier association. Before voting at *FutureDAO* in *Presentation* ①, Braid requires Alice to cryptographically associate id_1 and id_2 on the blockchain, forming a unified *associated identifier*. She

computes aid := $Hash(id_1, id_2, nonce)$ and registers the associated identifier aid to the VDR $\mathcal{R}$. This operation remains anonymous—no one knows that aid belongs to Alice—yet it creates an immutable, publicly verifiable record proving that the owner of id_1 is the same as that of id_2.

Cross-context Sybil resistance. This mechanism progressively limits Sybil attacks by creating permanent linkages. Once Alice's identifiers are associated, they remain permanently bound together across the Braid ecosystem. If another application, like a DeFi protocol, also requires Alice to present credentials, she must integrate these new credential identifiers into her existing *aid*. Figure 2 illustrates this scenario where Alice interacts with a DeFi app *InsuraFi* and presents credentials cred_3 and cred_4 from identifiers id_1 and id_3 (*Presentation* ②). Since id_1 is already part of aid, Alice must add id_3 to her existing aid before proceeding.

Identifier association creates unprecedented cross-context Sybil resistance by preventing exploitation of information silos between campaigns. While attackers can generate unlimited identifiers, these are merely valueless *blank slates*—the truly scarce resource is *distinct sets of valuable credentials*. Braid's forced consolidation leverages this scarcity, as demonstrated in *Presentation* ③, where the attempt to reclaim airdrop with a different identifier id_2 is immediately detected through established cross-context linkage. This rapidly depletes an attacker's limited pool of valuable identities across multiple campaigns, making sustained Sybil attacks exponentially more expensive.

3.2.3 Anonymous presentation with non-transferability. Anonymity and non-transferability present a fundamental conflict. Verifying legitimate ownership becomes difficult when credentials are anonymous, creating opportunities for unauthorized transfers. Braid resolves this by allowing Alice to prove ownership without revealing the associated identifier aid or any individual identifier id.

Instead of presenting identifying information, Alice generates a zero-knowledge proof. This proof attests to *FutureDAO* that she possesses the secret key corresponding to the identifier to which her credential was issued. This is possible because a cryptographic commitment, binding her identifier to her control over the secret key, was recorded on-chain during the identifier registration process. By demonstrating knowledge of this secret key in an entirely anonymous fashion, Braid ensures that the ability to present the credential is non-transferable. This achieves robust non-transferability without compromising her privacy.

3.2.4 A safety net: trustless key recovery. Months later, Alice loses her id_1 private key. In traditional systems, this would permanently erase her developer reputation and associated assets. Braid provides a safety net by shifting from *atomic secrets* (single seed phrases) to *structural knowledge* of her associated identity, offering two complementary recovery mechanisms.

The primary mechanism, *threshold key-based recovery*, allows Alice to recover id_1 by proving threshold control of other identifiers within aid (e.g., private keys of id_2 and id_3). This mitigates single points of failure, as one compromised key cannot compromise her entire association.

For users with complex digital footprints, *entropy-based recovery* leverages the hidden composition of their associated identifiers as a high-entropy secret. Recovery becomes *reconstruction* rather than *memorization*, where Alice can index public on-chain records to prove knowledge of her aid structure without depending on any specific key.

These methods offer a strategic trade-off: one serves as the universally applicable mechanism with strong security, while the other provides a key-agnostic alternative for advanced users. Both approaches are *trustless*, eliminating reliance on trusted third parties or centralized intermediaries.

Functionality $\mathcal{F}_{\text{Braid}}$

Identifier Registration: On receiving $(\texttt{register}, \mathsf{sid})$ from a party $\mathcal{P}$, sample a sk and calculate $\mathsf{pk} := \mathsf{PKGen}(\mathsf{sk})$. Sample an id. If id has been recorded, sample one again. Send $(\texttt{register}, \mathsf{sid}, \mathsf{pk}, \mathsf{id})$ to the adversary $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, record $(\mathcal{P}, \mathsf{id}, \mathsf{pk}, \mathsf{sk})$, and send $(\texttt{registered}, \mathsf{sid}, \mathsf{id})$ to $\mathcal{P}$.

Identifier Association: On receiving $(\texttt{associate}, \mathsf{sid}, \{\mathsf{id}_i\}_\ell)$ from a party $\mathcal{P}$, retrieve $(\mathcal{P}, \mathsf{id}_i, \mathsf{pk}_i, \mathsf{sk}_i)$ for all $i \in [\ell]$. Halt if some id_i is marked *associated*. Calculate $\mathsf{aid} := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a := 0)$, and send $(\texttt{associate}, \mathsf{sid}, \mathsf{aid})$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, mark all id_i as *associated* and record $(\mathcal{P}, \mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, 0)$. Finally, return $(\texttt{associated}, \mathsf{sid}, \mathsf{aid})$ to $\mathcal{P}$.

Association Appending: On receiving $(\texttt{append}, \mathsf{sid}, \mathsf{aid}, \mathsf{id}_{\ell+1})$ from $\mathcal{P}$, retrieve $(\mathcal{P}, \mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$ and $(\mathcal{P}, \mathsf{id}_{\ell+1}, \cdot, \cdot)$. Halt if aid is *invalidated* or $\mathsf{id}_{\ell+1}$ is *associated*. Calculate $\mathsf{aid}' := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_{\ell+1}, u_a)$, and send $(\texttt{append}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, mark aid as *invalidated*, mark $\mathsf{id}_{\ell+1}$ as *associated*, and record $(\mathcal{P}, \mathsf{aid}', \mathsf{id}_1, \ldots, \mathsf{id}_{\ell+1}, u_a)$. Finally, return $(\texttt{appended}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{P}$.

Association Merging: On receiving $(\texttt{merge}, \mathsf{sid}, \mathsf{aid}_1, \mathsf{aid}_2)$ from $\mathcal{P}$, retrieve $(\mathcal{P}, \mathsf{aid}_1, \mathsf{id}_1, \ldots, \mathsf{id}_t, u_{a1})$ and $(\mathcal{P}, \mathsf{aid}_2, \mathsf{id}_{t+1}, \ldots, \mathsf{id}_\ell, u_{a2})$. Halt if aid_1 or aid_2 is *invalidated*. Set $u_a := \max(u_{a1}, u_{a2})$. Calculate $\mathsf{aid}' := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$, and send $(\texttt{merge}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, mark aid_1 and aid_2 as *invalidated*, and record $(\mathcal{P}, \mathsf{aid}', \mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$. Finally, return $(\texttt{merged}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{P}$.

Association Refreshing: On receiving $(\texttt{refresh}, \mathsf{sid}, \mathsf{aid})$ from a party $\mathcal{P}$, retrieve $(\mathcal{P}, \mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$. Halt if aid is *invalidated*. Set $u'_a := u_a + 1$. Calculate $\mathsf{aid}' := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u'_a)$, and send $(\texttt{refresh}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, mark aid as *invalidated*, and record $(\mathcal{P}, \mathsf{aid}', \mathsf{id}_1, \ldots, \mathsf{id}_\ell, u'_a)$. Finally, return $(\texttt{refreshed}, \mathsf{sid}, \mathsf{aid}')$ to $\mathcal{P}$.

Credential Presentation: On receiving $(\texttt{campaign}, \mathsf{sid}, \phi)$ from $\mathcal{V}$, wait for input $(\texttt{present}, \mathsf{sid}, \mathsf{aid}, \{\mathsf{cred}_i\}_n)$ from $\mathcal{H}$. For all $i \in [n]$, parse $\mathsf{cred}_i := (\mathsf{id}_i^{\mathcal{I}}, \sigma_i, \mathsf{id}_i'^{\mathcal{H}}, s_i)$, and retrieve $(\mathcal{H}, \mathsf{id}_i'^{\mathcal{H}}, \cdot, \mathsf{sk}_i'^{\mathcal{H}})$. Then, check:

- If $(\mathcal{H}, \mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$ is recorded and not *invalidated*.
- If $\mathsf{id}_i'^{\mathcal{H}} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$.
- If $(\texttt{presented}, \mathsf{sid}, \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\})$ is not recorded.
- If cred_i is valid, recorded, and non-*revoked*, and $\phi(s_i) = 1$ holds.
- If $(\mathcal{I}, \mathsf{id}_i^{\mathcal{I}}, \mathsf{pk}_i^{\mathcal{I}}, \cdot)$ is recorded and $\mathsf{VerSig}(\mathsf{pk}_i^{\mathcal{I}}, \sigma_i, (\mathsf{id}_i^{\mathcal{I}}, \mathsf{id}_i'^{\mathcal{H}}, s_i)) = 1$.

Halt if any check fails. Else, send $(\texttt{present}, \mathsf{sid}, \{\mathsf{id}_i^{\mathcal{I}}\}_n)$ to $\mathcal{S}$. On receiving ok from $\mathcal{S}$, record $(\texttt{presented}, \mathsf{sid}, \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\})$ and send $(\texttt{verified}, \mathsf{sid}, \mathcal{H})$ to $\mathcal{V}$.

Key Recovery: On receiving $(\texttt{recover}, \mathsf{sid}, \mathsf{aid}, \mathsf{id}, \{\mathsf{id}'\}_t, \{\mathsf{sk}'\}_t)$ from a party $\mathcal{P}$, check:

- If $(\mathcal{P}, \mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, \cdot)$ is recorded and not *invalidated*.
- If $\mathsf{id} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$ and $\{\mathsf{id}'\}_t \subset \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$.
- If $(\mathcal{P}, \mathsf{id}, \mathsf{pk}, \mathsf{sk})$ is recorded.
- If $(\mathcal{P}, \mathsf{id}'_j, \cdot, \mathsf{sk}'_j)$ is recorded and *associated* for all $j \in [t]$.

Halt if any check fails. Else, sample a sk'' and calculate $\mathsf{pk}'' := \mathsf{PKGen}(\mathsf{sk}'')$. Send $(\texttt{recover}, \mathsf{sid}, \mathsf{id}, \mathsf{pk}'')$ to $\mathcal{S}$. On receiving ok from $\mathcal{S}$, replace the recorded $(\mathcal{P}, \mathsf{id}, \mathsf{pk}, \mathsf{sk})$ with $(\mathcal{P}, \mathsf{id}, \mathsf{pk}'', \mathsf{sk}'')$ and return $(\texttt{recovered}, \mathsf{sid})$ to $\mathcal{P}$.

Figure 3: The ideal functionality for Braid.

3.3 Functionality

We employ an ideal functionality $\mathcal{F}_{\text{Braid}}$ to provide a comprehensive overview of Braid, as shown in Figure 3.

- In the *identifier registration* phase, a holder $\mathcal{H}$ requests to generate a new identifier id, along with its keypair $(\mathsf{pk}, \mathsf{sk})$.
- In the *identifier association* phase, $\mathcal{P}$ requests to associate multiple identifiers $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ belonging to it by hashing them together

Decentralized ledger $\mathcal{L}$

The ledger functionality maintains a Merkle tree T, a nullifier list N, and an identifier-public key map M. It interacts with an ideal-world adversary $\mathcal{S}$.

Register: On receiving $(\texttt{register}, \mathsf{sid}, k, v)$ from a party $\mathcal{P}$, send $(\texttt{register}, \mathsf{sid}, k, v)$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, if there is already a record $(k, \cdot)$ in M, update it to (k, v); else, record (k, v) in M. Send $(\texttt{registered}, \mathsf{sid})$ to $\mathcal{P}$.

Record: On receiving $(\texttt{record}, \mathsf{sid}, v)$ from a party $\mathcal{P}$, calculate $(T, r, \rho) \leftarrow \mathsf{Insert}(v, T)$ and send $(\texttt{record}, \mathsf{sid}, r, \rho, v)$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, record r and send $(\texttt{recorded}, \mathsf{sid}, r, \rho)$ to $\mathcal{P}$.

Deprecate: On receiving $(\texttt{invalidate}, \mathsf{sid}, v)$ from a party $\mathcal{P}$, send $(\texttt{invalidate}, \mathsf{sid}, v)$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, mark v as *invalidated* by appending it to N, and send $(\texttt{invalidated}, \mathsf{sid})$ to $\mathcal{P}$.

Retrieve: On receiving $(\texttt{retrieve}, \mathsf{sid}, k)$ from a party $\mathcal{P}$, send $(\texttt{retrieve}, \mathsf{sid}, k)$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, if (k, v) is recorded in M, send $(\texttt{retrieved}, \mathsf{sid}, v)$ to $\mathcal{P}$, else send $(\texttt{unrecorded}, \mathsf{sid})$.

Check: On receiving $(\texttt{check}, \mathsf{sid}, v)$ from a party $\mathcal{P}$, send $(\texttt{check}, \mathsf{sid}, v)$ to $\mathcal{S}$. Upon receiving ok from $\mathcal{S}$, if $v \in N$, send $(\texttt{invalid}, \mathsf{sid})$ to $\mathcal{P}$. Otherwise, if v is recorded, send $(\texttt{checked}, \mathsf{sid})$ to $\mathcal{P}$; else send $(\texttt{unrecorded}, \mathsf{sid})$.

Figure 4: The ledger functionality $\mathcal{L}$.

with a nonce u_a initially set to 0. The resulting associated identifier aid is recorded for later use, and all member id's are marked as *associated*, preventing their inclusion in other associations.

- In the *association update* phase, $\mathcal{P}$ requests to securely modify an existing association without revealing its structure. To concisely capture this dynamic lifecycle, $\mathcal{F}_{\text{BRAID}}$ encompasses three specific methods for this phase: *Association Appending* (adding a new identifier to aid), *Association Merging* (combining two existing associations), and *Association Refreshing* (incrementing the nonce u_a to generate a new aid, proactively breaking nullifier-based correlations across different campaigns).
- In the *credential presentation* phase, a holder $\mathcal{H}$ presents several credentials $\mathsf{cred}_1, \ldots, \mathsf{cred}_n$ to a verifier $\mathcal{V}$ in campaign sid. The validity of each credential is checked individually to ensure that it indeed meets the verification criteria ϕ specified by $\mathcal{V}$ and has not been revoked. To prevent reuse of an association in sid, all holder identifiers $\mathsf{id}^{\mathcal{H}}$ of credentials should be associated within the same aid, and the exact association state $\{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$ is recorded to ensure that this state appears only once in sid.
- In the *key recovery* phase, $\mathcal{P}$ requests to update the keypair of an id with a lost key by proving its knowledge about id and a certain association aid.

Decentralized ledger. Our protocol requires processing and maintaining states such as participant identifiers and public keys. To capture the properties and interfaces of the decentralized ledger, we incorporate a functionality $\mathcal{L}$ as shown in Figure 4. Throughout the protocol's operation, invocations to the decentralized ledger are modeled as interactions with $\mathcal{L}$, which UC-realizes the functionalities for updating and querying on-chain states.

3.4 Security properties

Our ideal functionality $\mathcal{F}_{\text{BRAID}}$ clearly ensures the following security properties. As our BRAID protocol implements this ideal functionality, it naturally achieves these same security properties in the real world.

- *Credential privacy*: $\mathcal{V}$ cannot learn anything beyond the fact that the claim s in cred satisfies $\phi(s) = 1$.
- *Identifier privacy*: $\mathcal{V}$ cannot ascertain any individual identifier id that constitutes a specific associated identifier aid.
- *Contextual Unlinkability*: When $\mathcal{H}$ presents the same underlying credential multiple times *after refreshing the relevant association and credential handles* (n_a *and* n_i^c), $\mathcal{V}$ cannot link these presentations from the protocol transcript even if it colludes with other malicious verifiers or issuers.
- *Sybil-resistance*: $\mathcal{H}$ cannot present credentials multiple times under a campaign sid, given the presented credentials belong to the same associated identifier aid.
- *Non-transferability*: $\mathcal{H}$ cannot transfer its credential to another $\mathcal{H}'$ and enable the latter to legally present it.
- *Key recovery security*: a malicious $\mathcal{H}'$ cannot impersonate $\mathcal{H}$ and modify the public key of an identifier that doesn't belong to it.

Definition of security. Consider a protocol Π_{BRAID} with access to the decentralized ledger functionality $\mathcal{L}$. The output of an environment $\mathcal{E}$ interacting with Π_{BRAID} and an adversary $\mathcal{A}$ on the security parameter $\lambda \in \mathbb{N}$ and input $z \in \{0,1\}^*$ is denoted as $\text{EXEC}^{\mathcal{L}}_{\Pi_{\text{BRAID}}, \mathcal{A}, \mathcal{E}}(\lambda, z)$. On the other hand, the output of $\mathcal{E}$ interacting with $\mathcal{F}_{\text{BRAID}}$ and an ideal-world adversary $\mathcal{S}$ is denoted as $\text{IDEAL}_{\mathcal{F}_{\text{BRAID}}, \mathcal{S}, \mathcal{E}}(\lambda, z)$. Then, we define the security of BRAID, whose protocol is detailed in Section 4.

Definition 1. Let $\lambda \in \mathbb{N}$ be a security parameter, and Π_{BRAID} be a protocol under the $\mathcal{L}$-hybrid model. We say Π_{BRAID} UC-realizes $\mathcal{F}_{BRAID}$ under the $\mathcal{L}$-hybrid model if for all p.p.t. adversary $\mathcal{A}$ interacting with Π_{BRAID}, there exists a p.p.t. simulator $\mathcal{S}$ interacting with $\mathcal{F}_{BRAID}$, such that

$$EXEC^{\mathcal{L}}_{\Pi_{BRAID}, \mathcal{A}, \mathcal{E}}(\lambda, z) \approx IDEAL_{\mathcal{F}_{BRAID}, \mathcal{S}, \mathcal{E}}(\lambda, z)$$

holds for all p.p.t. environments $\mathcal{E}$ and for all $z \in \{0,1\}^*$.

4 Main construction

In this section, we introduce the BRAID construction and protocol. We first describe the *identifier association* mechanism and demonstrate its application in Sybil-resistant credential presentation. Next, we illustrate our *trustless key recovery* mechanism. Finally, we analyze the security properties of the protocol, including a theoretical analysis of its Sybil resistance capabilities. Figures 5, 12, and 14 provide a complete formalized description of the BRAID protocol.

4.1 Identifier registration and association

BRAID provides a mechanism called *identifier association* that mandates $\mathcal{H}$ to provide a *proof of association* before presenting to $\mathcal{V}$ multiple credentials under different identifiers.

We note that $\mathcal{R}$ maintains an incremental Merkle tree (with leaf nodes of tags h or aid) and a nullifier list N. All these data structures are publicly accessible on the blockchain. In a highly concurrent blockchain environment, the global tree state updates frequently. Since local zero-knowledge proof generation takes time (e.g., several seconds), the specific Merkle root a user fetches to construct a proof might become outdated by the time their transaction is mined. To address this race condition, $\mathcal{R}$ caches the 100 most recent Merkle roots. This sliding window allows the contract to accept proofs constructed against any recently valid historical state, preventing gas-wasting transaction reverts and synchronization failures.

Identifier registration. $\mathcal{H}$ can arbitrarily register new identifiers. To do this, $\mathcal{H}$ first generates a key pair by sampling $\mathsf{sk} \leftarrow_{\$} \mathbb{Z}_p$

and computing $\mathsf{pk} := \mathsf{PKGen}(\mathsf{sk})$, then sends pk to $\mathcal{R}$. $\mathcal{R}$ samples $\mathsf{id} \leftarrow_{\$} \mathbb{Z}_p$ and persists the pair $(\mathsf{id}, \mathsf{pk})$, then returns id to $\mathcal{H}$.

Afterwards, $\mathcal{H}$ calculates a tag $h := H_a(\mathsf{id}, \mathsf{sk})$ and proves in zero-knowledge that:

❶ pk is the public key of sk, and
❷ h derives from id and sk.

Concretely, this emits a proof π_g for relation R_g below:

$$\mathsf{pk} = \mathsf{PKGen}(\mathsf{sk}) \ \wedge \ h = H_a(\mathsf{id}, \mathsf{sk}).$$

$\mathcal{H}$ sends (id, h, π_g) to $\mathcal{R}$, who verifies π_g and records h in the incremental Merkle tree maintained by itself.

Identifier association. $\mathcal{H}$ could associate multiple identifiers $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ belonging to it into an aid. Registration and association are asynchronous, which is designed to prevent any party from inferring the components of aid by monitoring the order in which identifiers are registered and associated.

To associate identifiers, $\mathcal{H}$ generates a nonce $u_a := 0$. We adopt the convention that the identifier vector $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ is always canonically (lexicographically) sorted before hashing to ensure deterministic aggregation. Then, $\mathcal{H}$ computes $\mathsf{aid} := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$. The nonce u_a is introduced to provide unlinkability, see Appendix A.1. To justify the association, $\mathcal{H}$ proves in zero-knowledge that:

❶ aid is the aggregation of $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with a nonce u_a,
❷ For each $i \in [\ell]$, h_i is recorded in the ledger,
❸ h_i derives from id_i and its private key sk_i, and
❹ n_i also derives from id_i and sk_i.

This emits a proof π_a for relation R_a:

$$\mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \wedge \mathsf{Auth}(r_i, h_i, \rho_i) = 1$$
$$\wedge \ h_i = H_a(\mathsf{id}_i, \mathsf{sk}_i) \ \wedge \ n_i = H_n(\mathsf{id}_i, \mathsf{sk}_i).$$

To clarify, ❷ and ❸ ensure that id_i was indeed obtained through registration and belongs to $\mathcal{H}$ who possesses sk_i, while ❹ establishes bindings for id_i and emits a nullifier n_i to $\mathcal{R}$ to implicitly mark identifiers that have been *associated*.

$\mathcal{H}$ sends $(\mathsf{aid}, \{r_i\}_\ell, \{n_i\}_\ell, \pi_a)$ to $\mathcal{R}$, who verifies π_a and confirms that n_i's are not recorded in N. If this is the case, $\mathcal{R}$ records aid and invalidates all n_i's to N, indicating that all id_i's are *associated*.

Update by appending. Braid facilitates updating previously submitted associated identifiers by enabling $\mathcal{H}$ to append new identifiers to an existing association.

Concretely, Braid allows $\mathcal{H}$ to append a new $\mathsf{id}_{\ell+1}$ to an aid. To this end, $\mathcal{H}$ computes an updated $\mathsf{aid}' := H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, \mathsf{id}_{\ell+1}, u_a)$, a nullifier $n_a := H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$ to invalidate the old aid, and another nullifier $n_{\ell+1} := H_n(\mathsf{id}_{\ell+1}, \mathsf{sk}_{\ell+1})$ to invalidate $\mathsf{id}_{\ell+1}$. After this, $\mathcal{H}$ proves in zero-knowledge that:

❶ aid is the association of $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with the nonce u_a,
❷ aid' is the association of $\mathsf{id}_1, \ldots, \mathsf{id}_\ell, \mathsf{id}_{\ell+1}$ with u_a,
❸ aid is recorded by the ledger and is not *blocklisted*, and
❹ $\mathsf{id}_{\ell+1}$ is recorded but not *associated*.

This emits a proof π_p for relation R_p:

$$\mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \wedge \mathsf{aid}' = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_{\ell+1}, u_a)$$
$$\wedge \ \mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1 \ \wedge \ n_a = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$$
$$\wedge \ \mathsf{Auth}(r_{\ell+1}, h_{\ell+1}, \rho_{\ell+1}) = 1$$
$$\wedge \ h_{\ell+1} = H_a(\mathsf{id}_{\ell+1}, \mathsf{sk}_{\ell+1}) \ \wedge \ n_{\ell+1} = H_n(\mathsf{id}_{\ell+1}, \mathsf{sk}_{\ell+1}).$$

$\mathcal{H}$ sends $(r_a, r_{\ell+1}, n_a, n_{\ell+1}, \mathsf{aid}', \pi_p)$ to $\mathcal{R}$, who verifies π_p and confirms that $r_a, r_{\ell+1}$ are recorded tree roots and $n_a, n_{\ell+1}$ not recorded in N. If this is the case, $\mathcal{R}$ invalidates $n_{\ell+1}$ and n_a, indicating that $\mathsf{id}_{\ell+1}$ is *associated*, and the old version of aid has been *deprecated*.

Update by merging. Braid also enables $\mathcal{H}$ to merge two association aid_1 and aid_2 into one. To this end, $\mathcal{H}$ computes aid' and two nullifiers n_{a1} and n_{a2} with respect to aid_1 and aid_2. Then, $\mathcal{H}$ proves in zero-knowledge that:

❶ aid_1 derives from $\mathsf{id}_1, \ldots, \mathsf{id}_t$ with the nonce u_{a1},
❷ aid_2 derives from $\mathsf{id}_{t+1}, \ldots, \mathsf{id}_\ell$ with the nonce u_{a2},
❸ aid_1 and aid_2 are recorded and not *blocklisted*, and
❹ aid derives from all $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with a nonce u'_a, which is the maximum of u_{a1} and u_{a2}.

This emits a proof π_m for relation R_m:

$$\mathsf{aid}_1 = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_t, u_{a1}) \wedge \mathsf{Auth}(r_{a1}, \mathsf{aid}_1, \rho_{a1}) = 1 \wedge$$
$$\mathsf{aid}_2 = H_a(\mathsf{id}_{t+1}, \ldots, \mathsf{id}_\ell, u_{a2}) \wedge \mathsf{Auth}(r_{a2}, \mathsf{aid}_2, \rho_{a2}) = 1 \wedge$$
$$n_{a1} = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_t, u_{a1}) \wedge n_{a2} = H_n(\mathsf{id}_{t+1}, \ldots, \mathsf{id}_\ell, u_{a2})$$
$$\wedge \ \mathsf{aid}' = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u'_a) \ \wedge \ u'_a = \max(u_{a1}, u_{a2}).$$

$\mathcal{H}$ sends $(r_{a1}, r_{a2}, n_{a1}, n_{a2}, u_{a1}, u_{a2}, \mathsf{aid}', \pi_m)$ to $\mathcal{R}$, who verifies π_m and confirms that r_{a1}, r_{a2} are recorded tree roots and n_{a1}, n_{a2} not invalidated in N. If this is the case, $\mathcal{R}$ invalidates n_{a1} and n_{a2}, indicating that aid_1 and aid_2 are both *deprecated*.

4.2 Credential presentation

Associated identifiers allow verifiers to require robust anti-Sybil proofs from credential holders. Specifically, verifiers can demand pre-association of all identifiers linked to presented credentials. This enables Braid to instantly identify repeated participation in the same campaign.

Anti-Sybil credential presentation. In a Sybil-sensitive campaign (e.g., an airdrop), the verifier $\mathcal{V}$ samples a session (campaign-wise) identifier $\mathsf{sid} \leftarrow_{\$} \mathbb{Z}_p$ and broadcasts it to all potential participants, along with a public predicate ϕ that indicates the verification criterion for accepting credentials.

To participate in sid with credentials $\mathsf{cred}_1, \ldots, \mathsf{cred}_n$, the holder $\mathcal{H}$ justifies that none of these credentials has been presented in campaign sid. Concretely, $\mathcal{H}$ proves in zero-knowledge that:

❶ For each $i \in [n]$, cred_i has $\mathsf{id}_i^{\mathcal{H}}$ as its holder identifier,
❷ For each $i \in [n]$, $\mathsf{id}_i^{\mathcal{H}}$ is a component of aid,
❸ aid is recorded by the ledger and is not *blocklisted*, and
❹ aid has not been *presented* in sid before.

Credential validity and authenticity. Braid supports selective disclosure of credentials, where $\mathcal{H}$ demonstrates that the credential cred_i presented satisfies the criterion ϕ. For each $i \in [n]$, $\mathcal{H}$ proves in zero-knowledge that:

❺ The claim s_i in cred_i satisfies $\phi(s_i) = 1$,
❻ cred_i is issued and signed by $\mathcal{I}_i$, and
❼ cred_i is not *revoked*.
❽ cred_i is committed and recorded by $\mathcal{I}_i$.

Non-transferable ownership. Braid ensures anonymity during the presentation while preventing the malicious transfer of credentials. It resolves this inherent dilemma by leveraging the above-established identity infrastructure. The key lies in ensuring

that $\mathcal{H}$ possesses the private key $\mathsf{sk}_i^{\mathcal{H}}$ corresponding to $\mathsf{id}_i^{\mathcal{H}}$. For each $i \in [n]$, $\mathcal{H}$ proves in zero-knowledge that:

❾ A tag h_i is recorded by the registry $\mathcal{R}$, and
❿ h_i derives from $\mathsf{id}_i^{\mathcal{H}}$ and $\mathsf{sk}_i^{\mathcal{H}}$, where $\mathsf{id}_i^{\mathcal{H}}$ is exactly the holder identifier of cred_i.

Combining the constraints. As described above, $\mathcal{H}$ needs to justify the satisfaction of 10 constraints across the above three parts. To clarify, for ❸ and ❹, $\mathcal{H}$ computes a global nullifier n_a for aid and a campaign-wise nullifier n_e with respect to aid in campaign sid. To justify ❼, $\mathcal{H}$ hashes cred_i into n_i^c along with a randomness u_i^c sampled by $\mathcal{H}$ during credential issuance.

Finally, $\mathcal{H}$ generates a proof π_c for relation R_c composed of constraints ❶–❿:

$$\begin{aligned}
&\mathsf{cred}_i = (\mathsf{id}_i^{\mathcal{I}}, \sigma_i, \mathsf{id}_i^{\mathcal{H}}, s_i) \ \wedge\ \mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)\\
&\wedge\ \mathsf{id}_i^{\mathcal{H}} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\} \ \wedge\ n_a = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)\\
&\wedge\ \mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1 \ \wedge\ n_e = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, \mathsf{sid})\\
&\wedge\ \phi(s_i) = 1 \ \wedge\ \mathsf{VerSig}\big(\mathsf{pk}_i^{\mathcal{I}}, \sigma_i, (\mathsf{id}_i^{\mathcal{I}}, \mathsf{id}_i^{\mathcal{H}}, s_i)\big) = 1\\
&\wedge\ c_i^c = H_a(\mathsf{cred}_i, u_i^c) \ \wedge\ n_i^c = H_n(\mathsf{cred}_i, u_i^c)\\
&\wedge\ h_i = H_a(\mathsf{id}_i^{\mathcal{H}}, \mathsf{sk}_i^{\mathcal{H}}) \ \wedge\ \mathsf{Open}(\mathsf{sk}_i^{\mathcal{H}}, c_i^s, u_i^s) = 1\\
&\wedge\ \mathsf{Auth}(r_i, h_i, \rho_i) = 1 \ \wedge\ \mathsf{Auth}(r_i^c, c_i^c, \rho_i^c) = 1.
\end{aligned}$$

where VerSig is the signature verification algorithm.

$\mathcal{H}$ sends $(\{c_i^s\}_n, \{u_i^s\}_n, \{n_i^c\}_n, \{\mathsf{id}_i^{\mathcal{I}}\}_n, \{r_i\}_n, \{r_i^c\}_n, r_a, n_a, n_e, \pi_c)$ to $\mathcal{V}$, who resolves $\mathsf{pk}_i^{\mathcal{I}}$ from the VDR using $\mathsf{id}_i^{\mathcal{I}}$. Then, $\mathcal{V}$ confirms that r_a and all r_i are recorded tree roots, and that n_a is not invalidated. Also, $\mathcal{V}$ queries the credential management contract of $\mathcal{I}_i$ with r_i^c and n_i^c to confirm that all cred_i are issuer-recorded and not *revoked*. This asynchronous, ledger-based check ensures that issuers do not need to remain online during credential presentations.

Finally, $\mathcal{V}$ verifies π_c, and confirms that (sid, n_e) is not recorded. If all checks pass, $\mathcal{V}$ records (sid, n_e), indicating that aid has been presented in sid, and any further presentation in sid using the same aid will be detected by $\mathcal{V}$ and rejected. This also implies that if $\mathcal{H}$ presents multiple credentials in a campaign, it must have already formed an aid between the holder identifiers id_i of these credentials.

On-chain privacy boundaries and sequential ordering. To explicitly clarify the privacy guarantees and the underlying anti-Sybil mechanics during credential presentation, we delineate the strict boundary between public and private data:

- *Data published on-chain:* The protocol-level public values revealed to $\mathcal{V}$ are the verification transcript $(\{c_i^s\}_n, \{u_i^s\}_n, \{n_i^c\}_n, \{\mathsf{id}_i^{\mathcal{I}}\}_n, \{r_i\}_n, \{r_i^c\}_n, r_a, n_a, n_e, \pi_c)$. In smart-contract deployments, this transcript may be visible as calldata, whereas the presentation step permanently records only (sid, n_e). The transcript discloses issuer identifiers, roots, commitments, commitment randomness, nullifiers $(n_e, n_a, \{n_i^c\})$, and the proof π_c. The nullifiers and commitments appear pseudo-random, while issuer identifiers and roots serve as public verification metadata.
- *Data hidden in the presentation proof:* The holder identifiers $\mathsf{id}_i^{\mathcal{H}}$, private keys $\mathsf{sk}_i^{\mathcal{H}}$, undisclosed claims and signatures in cred_i, Merkle paths $(\rho_a, \rho_i, \rho_i^c)$, and the association witness $(\mathsf{aid}, \{\mathsf{id}_j\}, u_a)$ are encapsulated as private circuit witnesses. Although aid is a public leaf inserted during association, π_c hides which recorded aid leaf is used and which identifiers compose it.

Crucially, this enforces a strict *sequential ordering invariant*. The constraint $\mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1$ dictates that $\mathcal{H}$ must finalize the association aid on the ledger *before* generating the presentation proof. Utilizing the ledger as a chronological anchor binds each presentation to a pre-existing association state and prevents ephemeral, unrecorded associations from bypassing the Sybil check.

Updating after verification. A notable attack vector involves $\mathcal{H}$ participating in campaign sid with an anti-Sybil proof using aid, then updating to aid′ after the interaction ends to participate again. This creates a different but valid campaign-wise nullifier, circumventing Braid's anti-Sybil protection.

To counter this, $\mathcal{V}$ can defer verification of proof π_c until campaign sid concludes, ensuring any updated nullifiers n_a would already appear in $\mathcal{R}$'s nullifier set N. While effective for airdrops and DAO voting, this approach is less suitable for scenarios requiring immediate authentication. We present a solution for immediate verification in Appendix A.4.

Nullifier-based correlation. Multiple presentations of the same aid and cred generate identical nullifiers n_a and n_c, creating a *linkability risk*. A curious $\mathcal{V}$ could correlate different claims from the same $\mathcal{H}$ without knowing their actual identity. We address this privacy concern in Appendices A.1 and A.2.

Accountability and proof integrity. Beyond Sybil resistance, Braid provides accountability through *credential revocation* and *identifier blocklisting*. The identifier association mechanism delivers superior accountability compared to previous systems by extending penalties from a single identifier to its entire association. Braid also prevents *proof-forwarding* through designated-verifier proofs. Appendix A provides detailed explanations of these features.

4.3 Key recovery

Identifier association serves dual purposes: countering Sybil attacks and providing robust key recovery. While traditional key management depends on a *single atomic secret* (whose loss means irreversible control loss), Braid shifts to *provable structural knowledge*: demonstrating control over components that make up a user's aggregated identity aid.

This reframes recovery from *'Do you remember a secret?'* to *'Can you prove you still control parts of your identity?'* Braid offers two trustless recovery modes balancing security, usability, and identity complexity.

Threshold key-based recovery. This is Braid's default recovery mechanism, providing strong security against partial key compromise. The core principle: to recover a lost key for one identifier, $\mathcal{H}$ must prove control over a *threshold* t of other identifiers within the same association.

When $\mathcal{H}$ loses key sk for id, it can select t distinct identifiers $\{\mathsf{id}'_1, \ldots, \mathsf{id}'_t\}$ from the same aid, and demonstrate to $\mathcal{R}$ its knowledge of corresponding keys $\{\mathsf{sk}'_1, \ldots, \mathsf{sk}'_t\}$. To recover, $\mathcal{H}$ samples a new private key $\mathsf{sk}'' \leftarrow_\$ \mathbb{Z}_p$ and computes $\mathsf{pk}'' := \mathsf{PKGen}(\mathsf{sk}'')$, along with a tag $h'' := H_a(\mathsf{id}, \mathsf{sk}'')$ and a nullifier $n'' := H_n(\mathsf{id}, \mathsf{sk}'')$. $\mathcal{H}$ proves in zero-knowledge that:

❶ aid derives from $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with the nonce u_a,
❷ aid is recorded by the ledger and not *blocklisted*,
❸ pk'' is the corresponding public key of sk'',
❹ Both h'' and n'' derive from id and sk'',

❺ id and $\{\mathsf{id}'_1, \ldots, \mathsf{id}'_t\}$ are all components of aid, and
❻ For all id′ in $\{\mathsf{id}'_1, \ldots, \mathsf{id}'_t\}$, h' derives from id′ and its sk′.

This emits a proof π_k for relation R_k:

$$\begin{aligned}&\mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \ \wedge \ \mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1\\ &\wedge \ n_a = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \ \wedge \ \mathsf{id} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}\\ &\wedge \ \{\mathsf{id}'_1, \ldots, \mathsf{id}'_t\} \subset \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\} \ \wedge \ \mathsf{pk}'' = \mathsf{PKGen}(\mathsf{sk}'')\\ &\wedge \ \forall j \in [t] : \left(h'_j = H_a(\mathsf{id}'_j, \mathsf{sk}'_j) \ \wedge \ \mathsf{Auth}(r', h'_j, \rho'_j) = 1\right)\\ &\wedge \ h'' = H_a(\mathsf{id}, \mathsf{sk}'') \ \wedge \ n'' = H_n(\mathsf{id}, \mathsf{sk}'').\end{aligned}$$

$\mathcal{H}$ sends $(r', r_a, n_a, n'', h'', \mathsf{id}, \mathsf{pk}'', \pi_k)$ to $\mathcal{R}$. $\mathcal{R}$ verifies π_k and checks the on-chain state (that n_a and n'' are not nullified, and r and r_a are recorded roots). If valid, $\mathcal{R}$ records h'' and $(\mathsf{id}, \mathsf{pk}'')$, and invalidates n'' to N.

Entropy-based recovery. For $\mathcal{H}$ with a large and complex digital identity, BRAID offers a key-agnostic recovery option. When an aid contains a sufficiently large number of identifiers, the exact composition of aid, represented as the canonically ordered identifier vector used in the association, becomes a high-entropy secret known only to $\mathcal{H}$. In this mode, recovery shifts from proving key control to demonstrating knowledge of this structure. Users can retrieve public blockchain records to reconstruct this proof, eliminating the need to memorize complex details.

Concretely, to recover the key for $\mathsf{id} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$, $\mathcal{H}$ proves in zero-knowledge that:

❶ aid derives from $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with the nonce u_a,
❷ aid is recorded by the ledger and not *blocklisted*,
❸ id is a component of aid, i.e., $\mathsf{id} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}$,
❹ pk″ is the corresponding public key of sk″, and
❺ Both h'' and n'' derive from id and sk″.

This emits a proof π'_k for relation R'_k:

$$\begin{aligned}&\mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \ \wedge \ \mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1\\ &\wedge \ n_a = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \ \wedge \ \mathsf{id} \in \{\mathsf{id}_1, \ldots, \mathsf{id}_\ell\}\\ &\wedge \ \mathsf{pk}'' = \mathsf{PKGen}(\mathsf{sk}'')\\ &\wedge \ h'' = H_a(\mathsf{id}, \mathsf{sk}'') \ \wedge \ n'' = H_n(\mathsf{id}, \mathsf{sk}'').\end{aligned}$$

For these operations to maintain adequate security, $\mathcal{H}$ must ensure that aid contains more than q identifiers, where n is the total number of leaves in Merkle tree T. With $n \gg q$, the probability of an adversary satisfying relation R'_k is approximately $p \approx 1/n^q$. Security requires that $p < 1/2^\lambda$, comparable to the probability of guessing a private key for an identifier.

4.4 Security

In the subsequent analysis, we examine the security properties of protocol Π_{BRAID} by motivating the underlying mechanisms that enable it to fulfill the security requirements delineated in Section 3.4.

Credential and identifier privacy. Privacy is ensured by limiting the verifier $\mathcal{V}$'s knowledge to only the satisfaction of $\phi(s) = 1$ for s in cred. Concretely, $\mathcal{V}$ cannot obtain extra information about cred or identify individual id in an aid. This is guaranteed by the zero-knowledge property of π_c, which ensures that $\mathcal{V}$ gains no knowledge about the witness beyond what is stated in R_c. For identifier privacy, we consider the following two scenarios:

- During *identifier association*, the nullifier n_i represents the sole potential source of information facilitating the deduction of aid's components, while the one-wayness of H_n ensures that id_i cannot be reversed from n_i.
- During *credential presentation*, $\mathcal{V}$ is computationally incapable of reversing any id constituting aid or n_a, given the collision-resistance of H_a and H_n.

Sybil-resistance. $\mathcal{H}$ cannot submit cred's under the same aid within multiple interactions of sid. To illustrate this, assume that $\mathcal{H}$ presents cred_1 in one interaction and cred_2 in another, but the holder identifiers $\mathsf{id}_1^{\mathcal{H}}$ and $\mathsf{id}_2^{\mathcal{H}}$ of both credentials have been incorporated into the aid. To accomplish this, $\mathcal{H}$ would need to construct a π_c in the second interaction using a n'_e different from the n_e in the former interaction. This is computationally infeasible unless the holder compromises the soundness of π_c. Formally, we have the following theorem, with proof provided in Appendix C.

THEOREM 1. *Consider a holder $\mathcal{H}$ who participates in q consecutive campaigns $\mathsf{sid}_1, \ldots, \mathsf{sid}_q$, each of which requires $\mathcal{H}$ to present at least k credentials meeting specific predicate requirements. Let W be the event that $\mathcal{H}$ successfully launches a Sybil attack at the q-th campaign with a valid proof to the verifier $\mathcal{V}$. Then, we have:*

$$\Pr[W] \le \binom{s(q)}{k} \cdot \left(\frac{\ell(q) - 1}{\ell(q)^{2k-1}}\right) + \frac{q^2 \cdot \ell(q)^2}{2^\lambda},$$

where $s(q)$ is the number of credentials satisfying the predicates at the q-th campaign, and $\ell(q)$ is the number of unique identifiers possessed by $\mathcal{H}$ at the q-th campaign. We assume $s(q) = \mathsf{poly}(\lambda)$ and $\ell(q) = \mathsf{poly}(\lambda)$.

Key recovery. The capability to update a keypair associated with a specific id is restricted exclusively to its legitimate owner. This constraint is rooted in the fact that only the rightful owner possesses knowledge of the aid into which the id has been incorporated, as well as the private key sk′ corresponding to another id′ within that aid. The probability of any unauthorized entity generating a valid proof π_k and successfully convincing $\mathcal{R}$ is negligible.

Unlinkability. BRAID provides contextual unlinkability rather than global unlinkability. The proof π_c hides holder identifiers, undisclosed claims, Merkle paths, and the selected association witness. However, public nullifiers remain equality handles: reusing the same association state repeats n_a, and reusing the same credential state repeats n_i^c. Thus, presentations are unlinkable only after the relevant handles have changed. The public roots used for membership checks are registry states and do not by themselves reveal which credential or association was used.

Non-transferability. $\mathcal{H}$ without the private key $\mathsf{sk}^{\mathcal{H}}$ corresponding to the holder identifier $\mathsf{id}^{\mathcal{H}}$ of the credential cred cannot legitimately present it. The relation R_c incorporates a tag h derived from $\mathsf{sk}^{\mathcal{H}}$ into the ledger's Merkle tree. Given the soundness of π_c and collision resistance of the underlying hash function of the Merkle construct, the probability of constructing a valid π_c without $\mathsf{sk}^{\mathcal{H}}$ is negligible.

Formal security definition. To formally define the security of protocol Π_{BRAID}, we use the UC security definition introduced in Section 3.

THEOREM 2. *There exists a multi-party protocol Π_{BRAID} that UC-realizes the ideal functionality $\mathcal{F}_{\text{BRAID}}$ under the $\mathcal{L}$-hybrid model.*

Protocol Π_{Braid}

This protocol runs between the issuer $\mathcal{I}$, the credential holder $\mathcal{H}$, the verifier $\mathcal{V}$, and the verifiable registry $\mathcal{R}$. Parties in this protocol have access to the decentralized ledger functionality $\mathcal{L}$. In this functionality, we use $\mathcal{P}$ to refer to any possible entity other than $\mathcal{R}$.

Identifier Registration: On receiving $(\texttt{register}, \text{sid})$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ samples $\text{sk} \leftarrow_{\$} \mathbb{Z}_p$, calculates $\text{pk} := \text{PKGen}(\text{sk})$, and then sends pk to $\mathcal{R}$.
(2) $\mathcal{R}$ samples $\text{id} \leftarrow_{\$} \mathbb{Z}_p$, and sends $(\texttt{register}, \text{sid}, \text{id}, \text{pk})$ to $\mathcal{L}$. On receiving $(\texttt{registered}, \text{sid})$ from $\mathcal{L}$, $\mathcal{R}$ returns id to $\mathcal{P}$.
(3) $\mathcal{P}$ records $(\text{id}, \text{pk}, \text{sk})$, computes $h := H_a(\text{id}, \text{sk})$, and generates a π_g for relation R_g. $\mathcal{P}$ sends (h, π_g) to $\mathcal{R}$.
(4) $\mathcal{R}$ verifies π_g and aborts if verification fails. Else, $\mathcal{R}$ sends $(\texttt{record}, \text{sid}, h)$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r, \rho)$ from $\mathcal{L}$, $\mathcal{R}$ sends (r, ρ) to $\mathcal{P}$.
(5) $\mathcal{P}$ records (id, h, r, ρ), and then outputs $(\texttt{registered}, \text{sid}, \text{id})$.

Identifier Association: On receiving $(\texttt{associate}, \text{sid}, \{\text{id}_i\}_\ell)$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ checks if $(\text{id}_i, \text{pk}_i, \text{sk}_i)$ and $(\text{id}_i, h_i, r_i, \rho_i)$ are recorded for all $i \in [\ell]$. If not, $\mathcal{P}$ aborts. Else, $\mathcal{P}$ computes $\text{aid} := H_a(\text{id}_1, \ldots, \text{id}_\ell, u_a := 0)$ and $n_i := H_n(\text{id}_i, \text{sk}_i)$ for all i. $\mathcal{P}$ generates a proof π_a for relation R_a. Finally, $\mathcal{P}$ sends $(\text{aid}, \{r_i\}_\ell, \{n_i\}_\ell, u_a, \pi_a)$ to $\mathcal{R}$.
(2) $\mathcal{R}$ verifies π_a and aborts if verification fails. For all $i \in [\ell]$, $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, r_i)$ to $\mathcal{L}$. If $\mathcal{L}$ returns $(\texttt{unrecorded}, \text{sid})$ for any i, $\mathcal{R}$ aborts.
(3) For all $i \in [\ell]$, $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, n_i)$ to $\mathcal{L}$. If $\mathcal{L}$ returns $(\texttt{invalid}, \text{sid})$ for any i, $\mathcal{R}$ aborts.
(4) $\mathcal{R}$ sends $(\texttt{record}, \text{sid}, \text{aid})$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r_a, \rho_a)$ from $\mathcal{L}$, $\mathcal{R}$ sends (r_a, ρ_a) to $\mathcal{P}$, and sends $(\texttt{invalidate}, \text{sid}, n_i)$ for all $i \in [\ell]$ to $\mathcal{L}$.
(5) $\mathcal{P}$ records $(\text{aid}, r_a, \rho_a)$ and $(\text{aid}, \text{id}_1, \ldots, \text{id}_\ell, u_a)$, and then outputs $(\texttt{associated}, \text{sid}, \text{aid})$.

Credential Presentation: On receiving $(\texttt{campaign}, \text{sid}, \phi)$ and $(\texttt{present}, \text{sid}, \text{aid}, \{\text{cred}_i\}_n)$, $\mathcal{H}$ performs as follows:

(1) For all $i \in [n]$, $\mathcal{H}$ parses $\text{cred}_i := (\text{id}_i^{\mathcal{I}}, \sigma_i, \text{id}_i^{\mathcal{H}}, s_i)$. Then, $\mathcal{H}$ sends $(\texttt{retrieve}, \text{sid}, \text{id}_i^{\mathcal{I}})$ to $\mathcal{L}$, receiving $(\texttt{retrieved}, \text{sid}, \text{pk}_i^{\mathcal{I}})$.
(2) $\mathcal{H}$ retrieves $(\text{aid}, \text{id}_1, \ldots, \text{id}_\ell, u_a)$ and $(\text{aid}, r_a, \rho_a)$ from its record, along with $(\text{cred}_i, u_i^c, c_i^c, r_i^c, \rho_i^c)$, $(\text{id}_i^{\mathcal{H}}, \cdot, \text{sk}_i^{\mathcal{H}})$ and $(\text{id}_i^{\mathcal{H}}, h_i, r_i, \rho_i)$ for all $i \in [n]$. Then, $\mathcal{H}$ computes $n_a := H_n(\text{id}_1, \ldots, \text{id}_\ell, u_a)$ and $n_e := H_n(\text{id}_1, \ldots, \text{id}_\ell, \text{sid})$. For all $i \in [n]$, $\mathcal{H}$ computes $(c_i^s, u_i^s) := \text{Commit}(\text{sk}_i^{\mathcal{H}})$ and $n_i^c := H_n(\text{cred}_i, u_i^c)$. Finally, $\mathcal{H}$ generates a proof π_c for relation R_c, and sends $(\{c_i^s\}_n, \{u_i^s\}_n, \{n_i^c\}_n, \{\text{id}_i^{\mathcal{I}}\}_n, \{r_i\}_n, \{r_i^c\}_n, r_a, n_a, n_e, \pi_c)$ to $\mathcal{V}$.
(3) $\mathcal{V}$ sends $(\texttt{check}, \text{sid}, r_a)$ and $(\texttt{check}, \text{sid}, n_a)$ to $\mathcal{L}$, along with $(\texttt{check}, \text{sid}, r_i)$ and $(\texttt{retrieve}, \text{sid}, \text{id}_i^{\mathcal{I}})$ for all $i \in [n]$. It also checks, via the contract of $\text{id}_i^{\mathcal{I}}$, that r_i^c is a valid credential-tree root and that n_i^c is not in the revocation list. On receiving checked for r_a and all r_i, unrecorded for n_a, successful issuer-contract checks for all (r_i^c, n_i^c), and $(\texttt{retrieved}, \text{sid}, \text{pk}_i^{\mathcal{I}})$ for all $\text{id}_i^{\mathcal{I}}$, $\mathcal{V}$ checks if (sid, n_e) is recorded and rejects $\mathcal{H}$ if this is the case. Finally, $\mathcal{V}$ verifies π_c and aborts if verification fails. Else, $\mathcal{V}$ records (sid, n_e), and outputs $(\texttt{verified}, \text{sid}, \mathcal{H})$.

Key Recovery: On receiving $(\texttt{recover}, \text{sid}, \text{aid}, \text{id}, \{\text{id}'\}_t, \{\text{sk}'\}_t)$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ retrieves $(\text{aid}, \text{id}_1, \ldots, \text{id}_\ell, u_a)$, $(\text{aid}, r_a, \rho_a)$ and $(\text{id}'_j, h'_j, r', \rho'_j)$ $(\forall j \in [t])$ from its record, and computes $n_a := H_n(\text{id}_1, \ldots, \text{id}_\ell, u_a)$. Then, $\mathcal{P}$ samples a new private key $\text{sk}'' \leftarrow_{\$} \mathbb{Z}_p$, and computes $\text{pk}'' := \text{PKGen}(\text{sk}'')$, $h'' := H_a(\text{id}, \text{sk}'')$ and $n'' := H_n(\text{id}, \text{sk}'')$. Finally, $\mathcal{P}$ generates a proof π_k for the relation R_k, and sends $(r', r_a, n_a, n'', h'', \text{id}, \text{pk}'', \pi_k)$ to $\mathcal{R}$.
(2) $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, r_a)$, $(\texttt{check}, \text{sid}, r')$, $(\texttt{check}, \text{sid}, n_a)$ and $(\texttt{check}, \text{sid}, n'')$ to $\mathcal{L}$. On receiving checked for r_a, r' and unrecorded for n_a, n'', $\mathcal{R}$ verifies π_k, and aborts if verification fails. Then, $\mathcal{R}$ sends $(\texttt{register}, \text{sid}, \text{id}, \text{pk}'')$, $(\texttt{record}, \text{sid}, h'')$ and $(\texttt{invalidate}, \text{sid}, n'')$ to $\mathcal{L}$. On receiving $(\texttt{registered}, \text{sid})$, $(\texttt{recorded}, \text{sid}, r'', \rho'')$ and $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{R}$ sends (r'', ρ'') to $\mathcal{P}$.
(3) $\mathcal{P}$ records $(\text{id}, h'', r'', \rho'')$, and outputs $(\texttt{recovered}, \text{sid})$.

Figure 5: Braid main protocol.

In Appendix D, we will prove Theorem 2, which states that the execution of protocol Π_{Braid} in the $\mathcal{L}$-hybrid model is indistinguishable from the execution of $\mathcal{F}_{\text{Braid}}$. We construct simulators to examine the simulation under different scenarios of party corruption.

5 Implementation and evaluation

In this section, we conduct a comprehensive performance evaluation and benchmark testing of Braid. Our goal is to answer the following key questions:

- **Micro-benchmarks**: What is the computational and gas overhead for Braid's core operations, such as *identifier association*, *key recovery*, and *credential presentation*?
- **End-to-end performance**: In a real-world deployment, what is the total latency for a user to complete a full interaction, from *proof generation* to *on-chain confirmation*?
- **System scalability**: How does the system perform under load? What is the achievable throughput for on-chain operations, and what are the primary bottlenecks?
- **Comparative Analysis:** How does Braid's performance compare to baseline approaches that offer weaker security features?

Experimental setup. Our experiments examine performance metrics for holders and verifiers, who serve as the participating entities, under the following experimental setups:

- The *holder* operates on an M2-featured Mac mini with 8 cores and 8GB of RAM. The network connection is limited to a bandwidth of 80Mbps downstream and 15Mbps upstream. This configuration emulates a consumer-grade device with moderate computational capabilities.
- The *verifier* runs on a desktop powered by an Intel Ultra 7-265K processor and 32GB of RAM. The network connection offers a throughput of 1Gbps. This setup emulates a workstation with superior computational power.

Implementation details. We implement the arithmetic circuit in Braid[4] and construct zero-knowledge proofs using the Plonk implementation provided by gnark [9, 22, 30], which uses BLS12-381 as the underlying elliptic curve and the KZG scheme for polynomial commitment [38]. Braid utilizes Poseidon as the commitment primitive, which is an SNARK-friendly sponge construction [33]. Specifically, we use a $t = 12$ Poseidon-128 with $R_F = 8$ and $R_P = 56$, referring to the instantiation of Plonky2 [44]. The height of the Merkle tree T maintained by the registry $\mathcal{R}$ is set to 32, which is consistent with the configuration in Tornado Cash [49, 66].

5.1 Micro-benchmarks of core operations

We first analyze the performance of individual cryptographic operations in Braid.

[4]Repository: https://anonymous.4open.science/r/Braid-D3D4

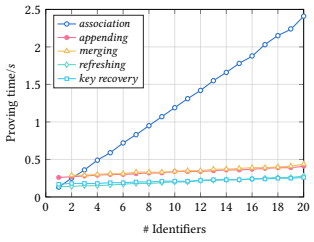

Figure 6: Proving time for association and key recovery.

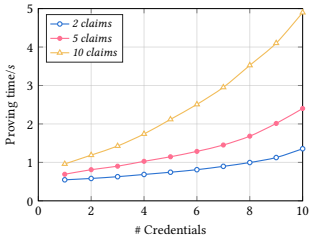

Figure 7: Proving time for credential presentation.

5.1.1 Identifier association and key recovery. We evaluate proof generation time for identifier association operations, including association, appending, merging, and refreshing. Figure 6 shows how proof time scales with the number of identifiers incorporated by the association.

Identifier generation and aggregation. The identifier generation operation's arithmetic circuit contains only 12,442 constraints on average, comprising a hash calculation and key derivation function evaluation, resulting in a 39ms proving time. As shown in Figure 6, the proving time for identifier association increases linearly with the number of incorporated identifiers. Intuitively, aggregating 10 registered identifiers into a single association generates an arithmetic circuit with 403,502 constraints, requiring approximately 1.19s for proving. When increased to 20 identifiers, the circuit grows to 0.8M constraints with a 2.41s proving time. Each additional identifier in the association adds about 40,350 constraints to the generated circuit, increasing the time cost by 120ms. Including the identifier generation time, each identifier adds approximately 160ms to the total processing time. This computational burden remains reasonable for most users with consumer-level devices of moderate capability.

Association update. The time cost of appending a new identifier to an existing association correlates directly with the number of existing identifiers in it. This correlation exists because the holder must evaluate the correct construction of aid in the circuit, with computational costs rising as identifier count increases. For instance, appending a new identifier to an association containing 20 identifiers generates an arithmetic circuit with 120,770 constraints, requiring approximately 410ms to process.

Merging two associations incurs higher costs than appending operations when comparing the final number of identifiers. This increased cost stems from the holder needing to evaluate the aid construction three times—a computationally intensive process. For instance, merging two associations of 10 identifiers each into a new 20-identifier association creates a circuit with 138,088 constraints, requiring 440ms of proving time.

Refreshing operations, by comparison, are relatively lightweight. They simply prove an implicit correlation between two associations from the u_a perspective. When refreshing the aid by incrementing u_a in a 20-identifier association, the process generates a circuit of 90,424 constraints and takes 280ms to prove.

Key recovery. The computational overhead for key recovery closely resembles that of association refreshing, since both operations require proving the construction of aid and its inclusion in T, plus some constant-level assertions. When recovering a key, it is essential that the holder's association contains enough identifiers to ensure adequate cryptographic entropy. For instance, with an aid containing 20 identifiers, the resulting circuit has 86,768 constraints and takes 0.26s to generate a proof.

Table 1: Gas consumption of interacting with the registry.

Operation	Gas Consumption
Identifier generation	313,656
Identifier association	689,739
Association appending	315,884
Association merging	335,421
Association refreshing	336,183
Key recovery	312,988

Table 2: Comparison of complexity with established schemes.

Scheme	Proof Size	Verifier Complexity
Braid	7 $\mathbb{G}_1$, 7 $\mathbb{F}_p$	12 $\mathbb{G}_1$-exp, 2 P
ZEBRA [51]	2 $\mathbb{G}_1$, 1 $\mathbb{G}_2$, 10 $\mathbb{F}_p$	3 $\mathbb{G}_1$-exp, 3 $\mathbb{G}_1$-op, 4 P
Coconut [55]	1 $\mathbb{G}_1$, 3 $\mathbb{G}_2$, (n+2) $\mathbb{F}_p$	(n+3) $\mathbb{G}_1$-exp, (n+3) $\mathbb{G}_1$-op, 2 $\mathbb{G}_2$-exp, 2 $\mathbb{G}_2$-op, 2 P
`zk-creds` [53]	2 $\mathbb{G}_1$, 1 $\mathbb{G}_2$	(n+1) $\mathbb{G}_1$-exp, 4 P

Gas costs and on-chain footprint. The on-chain costs of Braid are critical for its practicality. We deployed the verifier smart contract on a local Hardhat network emulating the Ethereum mainnet. Table 1 details the gas consumption for core operations when $\ell = 20$. Associating 20 identifiers costs approximately 690K gas, which translates to about $0.61 at a gas price of 0.20 Gwei. This is a one-time cost for establishing a strong identity base. Subsequent operations like refreshing or key recovery are more affordable, costing around 313K gas (~$0.28). We also measured the on-chain storage footprint. Each nullifier consumes 32 bytes of storage, and our contract stores the last 100 Merkle roots, occupying 3.2 KB.

5.1.2 Credential presentation and verification. For credential presentation, we evaluate the zero-knowledge proofs used in selective disclosure, examining their theoretical complexity and computational costs.

Theoretical complexity. Regarding credential verification, we compare the theoretical complexity of Braid with several recent milestone schemes, as illustrated in Table 2. Coconut and `zk-creds` have proof sizes and verification complexity that scale linearly with the number of claims n. Adding features like credential revocation further increases this verification complexity. While ZEBRA achieves constant-size proofs and verification complexity, its verification requires 4 pairing operations—significantly more time-consuming than Braid's 2 pairings. Braid integrates revocation natively without adding extra pairing operations to proof verification; the revocation-supporting relation only adds a credential-tree membership check inside the proved statement.

Holder overhead. During credential presentation, the holder calculates a proof for all cred_i in relation to R_c. The corresponding arithmetic circuit performs two main computations: the evaluation

of ϕ against s and signature verification, with the latter comprising about 95,000 constraints. Hash-based computations add another 71,000 constraints. The total number of constraints varies based on the number of elements in s and the complexity of ϕ. Figure 7 illustrates the computational overhead for holders generating proofs when presenting multiple credentials at once. The overhead varies with the number of claims per credential, with 5 credentials (5 claims each) taking about 1.14s.

Verifier overhead. The time consumed for credential verification in Braid remains constant regardless of credential complexity. This is evidenced by the verification process requiring only 2 pairing operations on BLS12-381 and 12 multiplications on $\mathbb{G}_1$, as outlined in Table 2. However, the overall verifier time does increase slightly with credential size due to statement processing overhead. For a presentation of 5 credentials containing 5 claims each, the total verification time is approximately 21ms.

5.2 End-to-end system performance

While micro-benchmarks are informative, end-to-end latency determines the real-world user experience. We measured the total time from the moment a holder initiates an action to its final confirmation on the blockchain.

Baseline Systems for Comparison. To contextualize Braid's performance and demonstrate the value of its security features, we implemented two baseline systems:

- **Baseline-Anon (anonymous credentials only):** This system represents a standard anonymous credential scheme without Sybil resistance. Holders present a zero-knowledge proof of credential ownership for a single credential. This baseline helps quantify the overhead introduced by Braid's core anti-Sybil features.
- **Baseline-SID (session-ID binding):** This system implements a naive approach where Sybil resistance within a single campaign is achieved by binding a credential to a session identifier, e.g., using a nullifier derived from $H(\mathsf{id}\|\mathsf{sid})$. This system cannot prevent cross-context Sybil attacks. Comparing Braid to Baseline-SID allows us to measure the cost of providing progressive, cross-context Sybil resistance.

Wall-clock latency. Figure 8 shows the end-to-end latency for a credential presentation requiring the association of 20 identifiers. The total time is approximately 14.3s. This is composed of: ZKP proof generation (1.2s), submitting the transaction to a public RPC endpoint (0.8s), transaction propagation and inclusion in a block (average 12s on Ethereum), and on-chain verification (0.3s). In contrast, *Baseline-SID* takes about 13.4s, with the difference primarily due to the smaller proof size. This demonstrates that Braid's stronger security guarantees introduce a moderate and acceptable increase in latency. Representing the lower bound of performance, *Baseline-Anon* is the fastest, with a total latency of about 12.9s. Its proof is the simplest, and its on-chain verification does not require any nullifier checks, resulting in slightly faster execution.

Communication overhead. The size of messages exchanged between parties impacts network latency and, in some cases, transaction costs. For a presentation of 3 credentials based on an aid of 10 identifiers, the total proof and public inputs sent by the holder to the verifier amount to 1,271 bytes. For the same scenario, *Baseline-SID*

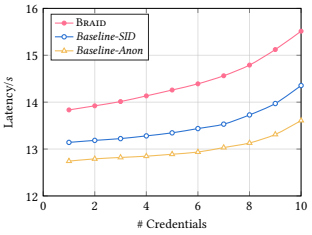

Figure 8: The E2E latency for an aid with 20 identifiers.

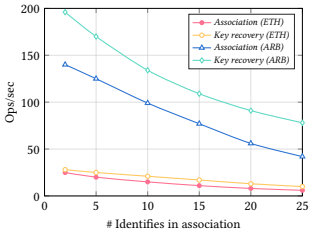

Figure 9: On-chain throughput on different platforms.

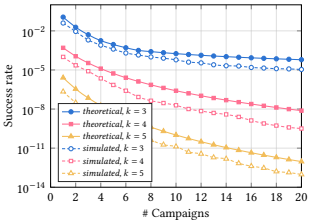

Figure 10: Sybil attack success rate with initial $\ell = 10$.

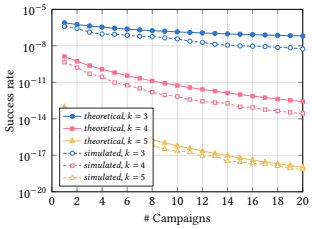

Figure 11: Sybil attack success rate with initial $\ell = 100$.

requires 890 bytes, as it needs to include the nullifier and related proofs. *Baseline-Anon* is the most lightweight, with a communication payload of only 780 bytes, since it only needs to prove the validity of the credential itself without any context-binding information.

Scalability and system throughput. We benchmarked the on-chain throughput of Braid's core operations, *identifier association* and *key recovery*, on both Ethereum and Arbitrum [3, 47]. As shown in Figure 9, the choice of platform is critical: deploying on Layer-2 provides a significant 5-7x throughput increase across all operations. Within each platform, the less state-intensive key recovery operation consistently outperforms identifier association. However, all curves exhibit a clear downward trend as aid complexity increases, confirming that on-chain computation—specifically sequential Merkle tree and nullifier updates—remains the fundamental bottleneck. While our implementation of caching the last 100 Merkle roots allows for parallel proof generation by users, the on-chain settlement remains the ultimate performance limiter. Nevertheless, as proven by the at-scale adoption of privacy-preserving protocols like Tornado Cash [49], this theoretical bottleneck does not pose a practical barrier on Layer-2 networks. Because each on-chain interaction incurs a minimal, constant-size footprint (e.g., storing a 32-byte nullifier) and users cover their individual gas fees, the operational overhead is naturally amortized, making Braid highly economically viable at scale.

Sybil resistance analysis. To evaluate Braid's resistance to Sybil attacks, we examined the probability of a malicious holder $\mathcal{H}$ successfully launching such attacks across a series of campaigns. Figures 10 and 11 show both theoretical and simulated probabilities for cases where $\mathcal{H}$ initially possesses 10 and 100 identifiers which have been associated. We analyzed scenarios requiring $k = 3, 4, 5$ qualifying credentials by verifiers per presentation. In our analysis

and simulation, we assumed $s(q)$ grows linearly with q, reflecting that $\mathcal{H}$ can dynamically obtain new credentials from issuers during the presentation process.

The results demonstrate that in all cases, $\mathcal{H}$'s success probability decreases rapidly with each additional campaign, eventually becoming negligible. Furthermore, when verifiers require more simultaneously presented credentials per event (i.e., increase the k value), the probability of $\mathcal{H}$'s credentials belonging to different identifiers increases. This compels faster identifier association, further diminishing the likelihood of successful Sybil attacks.

Crucially, in real-world deployments, the parameters $s(q)$ (number of valid credentials) and $\ell(q)$ (number of unique identifiers) operate on vastly different scales. Generating raw identifiers (ℓ) is instantaneous and cost-free, whereas acquiring valuable, issuer-verified credentials (s) requires significant time and capital. For instance, if an attacker spends months acquiring $s = 10$ credentials but spawns $\ell = 1000$ bot identifiers in seconds to launch an attack requiring just $k = 3$ credentials per presentation (a baseline in our evaluation), the attack success probability collapses to approximately 1.2×10^{-10}. Therefore, Braid's Sybil resistance is fundamentally anchored by the economic scarcity of credentials, substantially reducing the success probability and economic attractiveness of large-scale Sybil attacks under the stated assumptions.

6 Related work

To precisely position our contributions, it is essential to distinguish between *pure anonymous credential* (AC) schemes and comprehensive *decentralized identity* (DID) systems. AC schemes primarily serve as stateless cryptographic primitives, prioritizing absolute isolation for individual privacy-preserving claims. In contrast, DID systems like Braid construct a stateful, persistent identity layer to manage identity lifecycles, trustless key recovery, and ecosystem-wide accountability. Table 3 compares properties of Braid with those of mainstream solutions.

Decentralized identity. The Web3 ecosystem has embraced the *decentralized identity* framework for its potential to reduce identity breach risks inherent in traditional centralized systems [27, 46]. Various standards and specifications define the identification process and its participants [58]. At its core, this framework empowers users with autonomous management of their digital identities, free from reliance on centralized authorities or intermediaries. By harnessing technologies like blockchain, the framework strives to boost the interoperability of digital identities [40]. A similar concept is *soulbound tokens*, which use non-transferable, publicly verifiable digital tokens to represent digital assets that cannot be purchased, such as credentials, reputation, or interaction records [65].

Anonymous credential. Anonymous credentials, a field closely related to decentralized identity [5, 12–14, 17, 26], have evolved significantly since Chaum's pioneering work [21]. These schemes enable users to present identifier-linked claims while providing selective disclosure, identity unlinkability, and usage limitations [7, 24, 34, 51, 53, 55].

For instance, schemes like `zk-creds` [53] integrate zero-knowledge proofs directly into the credential structure to provide strong, global unlinkability. However, because pure AC schemes lack persistent identity infrastructure, they fundamentally struggle to prevent cross-context Sybil attacks or offer trustless key recovery. Preventing unauthorized credential sharing remains one of the most critical challenges in this research field. It intensifies in systems where anonymous credentials grant users significant autonomy, making it difficult to prevent voluntary credential transfers [37]. Although Camenisch et al. explored this issue, they did not provide an efficient solution [15]. Braid addresses this challenge by leveraging identity infrastructure to create non-transferable anonymous credentials, seamlessly bridging the gap between W3C-standard credentials and ZKP-based anonymity.

Table 3: Comparison of properties with established schemes.

Scheme	Type	*Sybil Resistance*	*Selective disclosure*	*Unlinkability*	*Credential Revocation*	*Identifier Blocklisting*
Braid	DID	●	●	◐	●	●
CanDID [43]	DID	◐	●	◐	●	○
Hades [63]	DID	◐	●	●	●	◐
ZEBRA [51]	AC	○	○	●	●	○
`zk-creds` [53]	AC	◐	●	●	●	○
Coconut [55]	AC	○	●	●	○	○

● for supporting properties, ○ for not supporting, and ◐ for partially supporting.
For Unlinkability, ● indicates traditional *global unlinkability* (absolute isolation across all contexts). ◐ denotes weaker system-level unlinkability: CanDID depends on its committee/MPC-based DID infrastructure, whereas Braid provides *contextual unlinkability* that holds only after the relevant public handles, such as n_a and n_i^c, have changed.

Sybil resistance in Web3. Sybil resistance is essential for all identity frameworks, particularly in Web3 scenarios like airdrops and DAOs where attackers can gain unfair advantages by creating multiple pseudonymous identities [2, 6, 41].

Researchers have proposed various solutions. Approaches like importing legacy profiles [43] or requiring offline gatherings [8] often impose substantial costs or logistical burdens [50, 54]. A more recent line of cryptographic work, such as Hades [63], enforces Sybil resistance within a campaign by binding users' identifiers for that specific context. While effective, this resistance is ephemeral and does not persist across different applications, allowing attackers to reuse their unlinked identities elsewhere. In contrast, Braid's identifier association mechanism creates persistent, cross-context linkages, fundamentally increasing the cost of long-term Sybil attacks while remaining non-intrusive for legitimate users.

7 Discussion and limitations

While Braid provides robust cross-context Sybil resistance and trustless key recovery, these security enhancements necessitate a pragmatic architectural trade-off, particularly regarding the theoretical bounds of identity unlinkability.

The contextual unlinkability limitation. Traditional pure anonymous credentials prioritize strict *global unlinkability*, ensuring that multiple presentations by the same user remain entirely isolated, even if executed simultaneously across different domains [15, 53]. Braid intentionally relaxes this property into *contextual unlinkability*. The primary limitation is handle reuse rather than concurrency itself: if a holder reuses the same association state, colluding verifiers can link presentations through the repeated nullifier n_a (see Appendix A.1); if the same credential state is reused, n_i^c provides an additional linking handle (see Appendix A.2).

Real-world trade-off and quantitative defense. This limitation is not a cryptographic flaw, but a deliberate design choice tailored for high-stakes decentralized applications [65]. In the Web3 ecosystem, the existential threat of *global Sybil attacks*, where attackers reuse the same farmed credentials across isolated information silos, vastly outweighs the privacy risks of linkability under handle reuse [25, 50]. We illustrate this necessity through two prominent mechanisms:

- **Quadratic Funding (e.g., Gitcoin Grants):** This mechanism allocates public matching funds mathematically based on the *number* of unique contributors rather than the total amount donated [11]. Consequently, attackers are highly incentivized to split their capital and spawn thousands of costless pseudonymous identifiers (ℓ). If absolute global unlinkability is preserved, an attacker can effortlessly exploit this formula to drain massive amounts of public goods funding [10, 32].
- **Ecosystem-wide Airdrops:** Networks distribute governance tokens to reward genuine early participants [62]. Qualifying for these rewards typically requires verifiable proof of sustained engagement (e.g., providing liquidity over months), making valid credentials inherently scarce and expensive to acquire (s) [6, 28].

Braid's contextual linkage substantially raises the economic cost of these attacks by translating Web3 economic scarcity into quantitative constraints on Sybil success. Consider an attacker attempting to exploit a Quadratic Funding round or a major Airdrop. As demonstrated in our Sybil resistance analysis (Section 5), the attacker can generate raw identifiers instantaneously and for free (e.g., $\ell = 1000$). However, due to the stringent requirements of credential issuers, the attacker might only possess a severely limited pool of valid, high-value credentials (e.g., $s = 10$).

If the system lacks cross-context linkage, the attacker can freely recycle these 10 credentials across their 1000 unlinked identities [41]. Braid fundamentally disrupts this by forcing malicious actors to cryptographically bind their scarce credentials to a single association. Subjected to the mathematical bounds of Theorem 1, when a verifier enforces a joint presentation policy requiring $k = 3$ distinct credentials per participant, the attacker's probability of successfully arming their bot network collapses to approximately 1.2×10^{-10}.

Therefore, contextual linkage should be understood as a quantitative and economic trade-off: it increases the cost of large-scale Sybil behavior while allowing honest holders to recover privacy across contexts by refreshing the relevant public handles. The resulting privacy loss is a controlled relaxation of global unlinkability under handle reuse.

8 Conclusion

We introduced Braid, a decentralized identity system designed to resolve three fundamental challenges in Web3: *pervasive Sybil attacks*, *catastrophic key loss*, and the conflict between *credential anonymity* and *non-transferability*.

Braid's core innovation, the *identifier association* mechanism, creates persistent, cross-context linkages that provide progressive Sybil resistance, overcoming the single-campaign limitations of prior work. This mechanism operates without the heavy burdens of collateral or physical gatherings. Furthermore, this on-chain structure enables a *trustless key recovery* paradigm based on provable structural knowledge, allowing users to recover assets without relying on trusted third parties. Finally, Braid achieves *non-transferability* for anonymous credentials and creates systematic disincentives against key-sharing, preventing malicious transfer of anonymous credentials. Comprehensive evaluation confirms Braid's practicality and effectiveness. By cohesively integrating these solutions, Braid offers a significant step towards a more *secure*, *resilient*, and *trustworthy* decentralized ecosystem.

References

[1] Nicholas Albrecht. 2021. Tens of billions worth of Bitcoin have been locked by people who forgot their key. https://www.nytimes.com/2021/01/13/business/tens-of-billions-worth-of-bitcoin-have-been-locked-by-people-who-forgot-their-key.html

[2] Diana Ambolis. 2023. Your Ultimate Guide To: Sybil Attacks In Decentralization. https://blockchainmagazine.net/your-ultimate-guide-to-sybil-attacks-in-decentralization/.

[3] Arbitrum. 2025. Arbitrum: Gas and Fees. https://docs.arbitrum.io/arbos/gas.

[4] James Austgen, Andrés Fábrega, Sarah Allen, Kushal Babel, Mahimna Kelkar, and Ari Juels. 2023. DAO Decentralization: Voting-Bloc Entropy, Bribery, and Dark DAOs. *arXiv preprint arXiv:2311.03530* 2311, 03530 (2023), 1–46.

[5] Foteini Baldimtsi and Anna Lysyanskaya. 2013. Anonymous Credentials Light. In *Proceedings of the 20th ACM SIGSAC conference on Computer and Communications Security (CCS 2013)*. ACM, Berlin, Germany, 1087–1098.

[6] Anna Baydakova. 2023. Sybil Millionaires: How Airdrop Hunters Trick Projects and Snatch Fortunes. https://www.coindesk.com/consensus-magazine/2023/04/10/crypto-airdrop-sybil-attacks/.

[7] Johannes Blömer, Jan Bobolz, Denis Diemert, and Fabian Eidens. 2019. Updatable Anonymous Credentials and Applications to Incentive Systems. In *Proceedings of the 26th ACM SIGSAC Conference on Computer and Communications Security (CCS 2019)*. ACM, London, United Kingdom, 1671–1685.

[8] Maria Borge, Eleftherios Kokoris-Kogias, Philipp Jovanovic, et al. 2017. Proof-of-Personhood: Redemocratizing Permissionless Cryptocurrencies. In *Proceedings of 2017 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW 2017)*. IEEE, Paris, France, 23–26.

[9] Gautam Botrel, Thomas Piellard, Youssef El Housni, Ivo Kubjas, and Arya Tabaie. 2025. gnark: v0.9.1. https://doi.org/10.5281/zenodo.5819104. doi: 10.5281/zenodo.5819104

[10] Alain Brenzikofer. 2023. Quadratic Voting for Polkadot Governance. https://forum.polkadot.network/t/quadratic-voting-for-polkadot-governance/3071.

[11] Vitalik Buterin, Zoë Hitzig, and E Glen Weyl. 2019. A flexible design for funding public goods. *Management Science* 65, 11 (2019), 5171–5187.

[12] Jan Camenisch, Manu Drijvers, and Maria Dubovitskaya. 2017. Practical UC-Secure Delegatable Credentials with Attributes and their Application to Blockchain. In *Proceedings of the 24th ACM SIGSAC Conference on Computer and Communications Security (CCS 2017)*. ACM, Dallas, Texas, USA, 683–699.

[13] Jan Camenisch, Maria Dubovitskaya, Kristiyan Haralambiev, and Markulf Kohlweiss. 2015. Composable and Modular Anonymous Credentials: Definitions and Practical Constructions. In *Proceedings of 21st International Conference on the Theory and Application of Cryptology and Information Security (AsiaCrypt 2015)*. Springer, Auckland, New Zealand, 262–288.

[14] Jan Camenisch, Susan Hohenberger, Markulf Kohlweiss, Anna Lysyanskaya, and Mira Meyerovich. 2006. How to Win the Clonewars: Efficient Periodic N-Times Anonymous Authentication. In *Proceedings of the 13th ACM conference on Computer and Communications Security (CCS 2006)*. ACM, Alexandria, Virginia, USA, 201–210.

[15] Jan Camenisch and Anna Lysyanskaya. 2001. An Efficient System for Non-Transferable Anonymous Credentials with Optional Anonymity Revocation. In *Proceedings of 20th Annual International Conference on the Theory and Applications of Cryptographic Techniques (EuroCrypt 2001)*. Springer, Innsbruck, Austria, 93–118.

[16] Jan Camenisch and Anna Lysyanskaya. 2002. Dynamic Accumulators and Application to Efficient Revocation of Anonymous Credentials. In *Proceedings of 22nd Annual International Cryptology Conference (Crypto 2002)*. Springer, Santa Barbara, California, USA, 61–76.

[17] Jan Camenisch and Anna Lysyanskaya. 2004. Signature Schemes and Anonymous Credentials from Bilinear Maps. In *Proceedings of 24th Annual International Cryptology Conference (Crypto 2004)*. Springer, Santa Barbara, California, USA, 56–72.

[18] Jan Camenisch, Anna Lysyanskaya, and Gregory Neven. 2012. Practical Yet Universally Composable Two-Server Password-Authenticated Secret Sharing. In *Proceedings of the 19th ACM conference on Computer and Communications Security (CCS 2012)*. ACM, Raleigh, North Carolina, USA, 525–536.

[19] Jan Camenisch and Markus Stadler. 1997. Efficient group signature schemes for large groups. In *Proceedings of 17th Annual International Cryptology Conference (Crypto 1997)*. Springer, Santa Barbara, California, USA, 410–424.

[20] Ran Canetti, Yehuda Lindell, Rafail Ostrovsky, and Amit Sahai. 2002. Universally Composable Two-Party and Multi-Party Secure Computation. In *Proceedings of the 34th annual ACM symposium on Theory of computing*. ACM, Montreal, Quebec, Canada, 494–503.

[21] David Chaum. 1985. Security without Identification: Transaction Systems to Make Big Brother Obsolete. *Commun. ACM* 28, 10 (1985), 1030–1044.

[22] Consensys. 2025. Consensys/gnark. https://github.com/Consensys/gnark.

[23] Rasmus Dahlberg, Tobias Pulls, and Roel Peeters. 2016. Efficient Sparse Merkle Trees: Caching Strategies and Secure (Non-) Membership Proofs. In *Proceedings of 21st Nordic Conference (NordSec 2016)*. Springer, Oulu, Finland, 199–215.

[24] Jack Doerner, Yashvanth Kondi, Eysa Lee, Abhi Shelat, and LaKyah Tyner. 2023. Threshold BBS+ Signatures for Distributed Anonymous Credential Issuance. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 773–789.

[25] John R Douceur. 2002. The sybil attack. In *International workshop on peer-to-peer systems*. Springer, Berlin, Germany, 251–260.

[26] Changlai Du, Hexuan Yu, Yang Xiao, Y Thomas Hou, Angelos D Keromytis, and Wenjing Lou. 2023. UCBlocker: Unwanted Call Blocking Using Anonymous Authentication. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX, Anaheim, CA, USA, 445–462.

[27] Paul Dunphy and Fabien AP Petitcolas. 2018. A First Look at Identity Management Schemes on the Blockchain. *IEEE security & privacy* 16, 4 (2018), 20–29.

[28] Sizheng Fan, Tian Min, Xiao Wu, and Wei Cai. 2023. Altruistic and Profit-Oriented: Making Sense of Roles in Web3 Community from Airdrop Perspective. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. ACM, Hamburg, Germany, 1–16.

[29] Bryan Ford. 2020. Identity and Personhood in Digital Democracy: Evaluating Inclusion, Equality, Security, and Privacy in Pseudonym Parties and other Proofs of Personhood. *arXiv preprint arXiv:2011.02412* 2011, 02412 (2020), 1–27.

[30] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. 2019. Plonk: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge. *Cryptology ePrint Archive* 2019, 953 (2019), 1–34.

[31] Edd Gent. 2023. A Cryptocurrency for the Masses or a Universal ID?: Worldcoin Aims to Scan all the World's Eyeballs. *IEEE Spectrum* 60, 1 (2023), 42–57.

[32] Gitcoin. 2022. Streamlining Sybil Defense for Gitcoin Grants with Gitcoin Passport. https://www.gitcoin.co/blog/streamlining-sybil-defense/.

[33] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, et al. 2021. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In *USENIX Security Symposium*, Vol. 2021. USENIX, Virtual Events, 519–535.

[34] Lucjan Hanzlik and Daniel Slamanig. 2021. With a Little Help from my Friends: Constructing Practical Anonymous Credentials. In *Proceedings of the 28th ACM SIGSAC Conference on Computer and Communications Security (CCS 2021)*. ACM, Virtual Event, Korea, 2004–2023.

[35] Shuangyu He, Qianhong Wu, Xizhao Luo, Zhi Liang, Dawei Li, Hanwen Feng, Haibin Zheng, and Yanan Li. 2018. A Social-Network-based Cryptocurrency Wallet-Management Scheme. *IEEE Access* 6 (2018), 7654–7663.

[36] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. 2014. Round-Optimal Password-Protected Secret Sharing and T-PAKE in the Password-Only Model. In *Proceedings of 20th International Conference on the Theory and Application of Cryptology and Information Security (AsiaCrypt 2014)*. Springer, Kaoshiung, Taiwan, ROC, 233–253.

[37] Saqib A Kakvi, Keith M Martin, Colin Putman, and Elizabeth A Quaglia. 2023. SoK: Anonymous Credentials. In *International Conference on Research in Security Standardisation*. Springer, Lyon, France, 129–151.

[38] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. 2010. Constant-Size Commitments to Polynomials and their Applications. In *Proceedings of 16th International Conference on the Theory and Application of Cryptology and Information Security (AsiaCrypt 2010)*. Springer, Singapore, 177–194.

[39] Nanak Nihal Khalsa, Caleb Tuttle, Lily Hansen-Gillis, et al. 2022. Holonym: A Decentralized Zero-Knowledge Smart Identity Bridge. https://holonym.io/whitepaper.pdf.

[40] Dmitry Khovratovich and Jason Law. 2023. Sovrin: Digital Identities in the Blockchain Era. https://sovrin.org/wp-content/uploads/AnonCred-RWC.pdf.

[41] Oliver Knight. 2023. Connext Airdrop Marred by $38K Sybil Bot Attack. https://www.coindesk.com/business/2023/09/05/connext-airdrop-marred-by-38k-sybil-bot-attack/.

[42] Yue Liu, Qinghua Lu, Hye-Young Paik, and Xiwei Xu. 2020. Design Patterns for Blockchain-based Self-sovereign Identity. In *Proceedings of 25th European Conference on Pattern Languages of Programs (EuroPLoP 20)*. ACM, Virtual Event, Germany, 1–14.

[43] Deepak Maram, Harjasleen Malvai, Fan Zhang, et al. 2021. CanDID: Can-Do Decentralized Identity with Legacy Compatibility, Sybil-Resistance, and Accountability. In *Proceedings of 2021 IEEE Symposium on Security and Privacy (S&P 2021)*. IEEE, San Francisco, CA, USA, 1348–1366.

[44] Mir-Protocol. 2025. Mir-Protocol/Plonky2. https://github.com/mir-protocol/plonky2.

[45] Kelsie Nabben. 2021. Is a Decentralized Autonomous Organization a Panopticon? Algorithmic governance as creating and mitigating vulnerabilities in DAOs. In *Proceedings of the Interdisciplinary Workshop on (de) Centralization in the Internet*. ACM, Virtual Event, Germany, 18–25.

[46] Nitin Naik and Paul Jenkins. 2020. uPort Open-Source Identity Management System: An Assessment of Self-Sovereign Identity and User-Centric Data Platform Built on Blockchain. In *Proceedings of 2020 IEEE International Symposium on Systems Engineering (ISSE 2020)*. IEEE, Vienna, Austria, 1–7.

[47] Nansen. 2025. Arbitrum Gas Tracker. https://pro.nansen.ai/multichain/arbitrum/gas-tracker.

[48] Charalampos Papamanthou, Roberto Tamassia, and Nikos Triandopoulos. 2011. Optimal Verification of Operations on Dynamic Sets. In *Proceedings of 31st Annual International Cryptology Conference (Crypto 2011)*. Springer, Santa Barbara, California, USA, 91–110.

[49] Alexey Pertsev, Roman Semenov, and Roman Storm. 2019. Tornado Cash Privacy Solution Version 1.4. *Tornado cash privacy solution version* 1 (2019), 1–3.

[50] Moritz Platt and Peter McBurney. 2021. Sybil Attacks on Identity-Augmented Proof-of-Stake. *Computer Networks* 199 (2021), 108424.

[51] Deevashwer Rathee, Guru Vamsi Policharla, Tiancheng Xie, Ryan Cottone, and Dawn Song. 2022. ZEBRA: Anonymous Credentials with Practical On-Chain Verification and Applications to KYC in DeFi. *Cryptology ePrint Archive* 2022, 1286 (2022), 1–21.

[52] RoboTeddy. 2021. Proof of Humanity: the Cost of Attack. https://hackmd.io/@RoboTeddy/SkFEYwptd.

[53] Michael Rosenberg, Jacob White, Christina Garman, and Ian Miers. 2023. zkcreds: Flexible Anonymous Credentials from zkSNARKs and Existing Identity Infrastructure. In *2023 IEEE Symposium on Security and Privacy (SP)*. IEEE, San Francisco, CA, USA, 790–808.

[54] Divya Siddarth, Sergey Ivliev, Santiago Siri, and Paula Berman. 2020. Who Watches the Watchmen? A Review of Subjective Approaches for Sybil-resistance in Proof of Personhood Protocols. *Frontiers in Blockchain* 3 (2020), 46.

[55] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2019. Coconut: Threshold Issuance Selective Disclosure Credentials with Applications to Distributed Ledgers. In *2019 Network and Distributed System Security Symposium (NDSS)*. NDSS, San Diego, California, USA, 1–15.

[56] Specops. 2023. The Genesis Market Takedown - Keep Users Credentials Secure. https://www.bleepingcomputer.com/news/security/the-genesis-market-takedown-keep-users-credentials-secure/.

[57] Manu Sporny, Dave Longley, and David Chadwick. 2025. *Verifiable Credentials Data Model 2.0*. World Wide Web Consortium (W3C). https://www.w3.org/TR/vc-data-model-2.0/

[58] Manu Sporny, Dave Longley, Markus Sabadello, et al. 2022. *Decentralized Identifiers (DIDs) 1.0*. World Wide Web Consortium (W3C). https://www.w3.org/TR/did-core/

[59] Caliendo Tom. 2023. Data Breaches and the Role of Stolen Credentials in 2023. https://www.secjuice.com/data-breaches-stolen-credentials-2023/.

[60] Alin Tomescu, Vivek Bhupatiraju, Dimitrios Papadopoulos, Charalampos Papamanthou, Nikos Triandopoulos, and Srinivas Devadas. 2019. Transparency Logs via Append-Only Authenticated Dictionaries. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS 2019)*. ACM, London, United Kingdom, 1299–1316.

[61] Nirvan Tyagi, Ben Fisch, Andrew Zitek, Joseph Bonneau, and Stefano Tessaro. 2022. VeRSA: Verifiable Registries with Efficient Client Audits from RSA Authenticated Dictionaries. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS 2022)*. ACM, Los Angeles, CA, USA, 2793–2807.

[62] Riad S. Wahby, Dan Boneh, Christopher Jeffrey, et al. 2020. An Airdrop that Preserves Recipient Privacy. In *Proceedings of 24th International Conference Financial Cryptography and Data Security (FC 2020)*. Springer, Kota Kinabalu, Malaysia, 444–463.

[63] Ke Wang, Jianbo Gao, Qiao Wang, Jiashuo Zhang, Yue Li, Zhi Guan, and Zhong Chen. 2023. Hades: Practical Decentralized Identity with Full Accountability and Fine-grained Sybil-resistance. In *Proceedings of the 39th Annual Computer Security Applications Conference* (Austin, TX, USA) *(ACSAC '23)*. ACM, New York, NY, USA, 216–228.

[64] Shuai Wang, Wenwen Ding, Juanjuan Li, et al. 2019. Decentralized Autonomous Organizations: Concept, Model, and Applications. *IEEE Transactions on Computational Social Systems* 6, 5 (2019), 870–878.

[65] E Glen Weyl, Puja Ohlhaver, and Vitalik Buterin. 2022. Decentralized Society: Finding Web3's Soul. http://dx.doi.org/10.2139/ssrn.4105763.

[66] Jingyan Yang, Shang Gao, Guyue Li, Rui Song, and Bin Xiao. 2022. Reducing Gas Consumption of Tornado Cash and other Smart Contracts in Ethereum. In *2022 IEEE International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom)*. IEEE, Wuhan, China, 921–926.

Ethical Considerations

This appendix provides a stakeholder-based ethics analysis for our work on Braid, following the guidelines set forth by the ACM CCS. We have considered the ethical principles of Beneficence, Respect for Persons, Justice, and Respect for Law and Public Interest as outlined in The Menlo Report.

Stakeholders. We identify the following primary stakeholders who may be impacted by our research and its publication:

- **Legitimate Web3 users**: Individuals participating in DAOs, airdrops, and other decentralized applications who desire fair systems and privacy.
- **System and application developers**: Engineers and organizations building decentralized systems who need practical tools to combat Sybil attacks and manage user identity.
- **Potential attackers/malicious actors**: Individuals or groups aiming to exploit decentralized systems for unfair gain (e.g., through Sybil attacks).
- **The research team**: The authors of this paper.
- **Society at large**: The broader community that may be positively or negatively affected by the proliferation of more robust (or potentially misused) decentralized identity technologies.

Ethical principles and impact analysis. **Beneficence.** Our work is primarily motivated by the principle of beneficence, aiming to introduce substantial, tangible benefits to the Web3 ecosystem by enhancing system fairness and user security, while carefully considering and mitigating the inherent risks of dual-use in privacy technologies.

- **Benefits**: Our research aims to provide significant benefits to legitimate users and developers. Braid offers a practical path toward fairer DAOs, more equitable airdrops, and a more trustworthy Web3 ecosystem by raising the cost of Sybil attacks. The trustless key recovery mechanism directly addresses the massive financial losses (estimated in the billions of dollars) from lost cryptocurrency keys, a tangible benefit to users.
- **Harms**: The primary ethical concern is the dual-use nature of privacy-enhancing technologies. While Braid is designed to protect legitimate users, the same strong anonymity and Sybil-resistance properties could potentially be leveraged by malicious actors to create robust, difficult-to-trace identities for illicit activities.
- **Mitigation**: To mitigate the risk of undiscovered flaws, our entire implementation, including smart contracts and ZKP circuits, is made publicly available for scrutiny (see Open Science appendix). We believe that the open review process is the most effective way to identify and fix potential issues. Regarding the dual-use concern, we argue that the core mechanism of Braid —linking identifiers based on the presentation of valuable, issuer-verified credentials—inherently discourages purely anonymous, untraceable actors. Gaining the credentials needed to build a powerful Braid identity requires interaction with credential issuers, providing a level of accountability.

Respect for persons. Our work strongly aligns with this principle. Braid is designed to enhance user autonomy by providing *self-sovereign control over identity* and a *trustless method for key recovery*, reducing dependence on centralized custodians. The use of zero-knowledge proofs is central to protecting user privacy, ensuring that individuals can prove their eligibility without revealing the specifics of their credentials or the structure of their associated identity.

Justice. Braid promotes justice by aiming to level the playing field in decentralized systems. By making Sybil attacks more difficult, it helps ensure that resources and governance rights are distributed more fairly based on *one person, one vote* principles, rather than being captured by those with the resources to create many fake identities. The system is permissionless and open-source, making its benefits accessible to all, not just a privileged few.

Respect for law and public interest. We conducted our research using standard cryptographic tools and public blockchain platforms. No live systems were tested without consent, no terms of service were violated, and no deception was used. Our work does not involve vulnerabilities in existing systems but rather proposes a new system architecture. We believe the publication of this work is in the public interest, as it contributes to a more secure and reliable decentralized ecosystem.

Decision to proceed and publish. Weighing the significant potential benefits—*fairer governance*, *enhanced user security*, and *greater financial safety* through key recovery—against the mitigated risk of dual-use, we concluded that the decision to conduct and publish this research is ethically sound. The potential harms are not unique to Braid but are inherent in most privacy-enhancing technologies. We believe that by tying identity strength to verifiable credentials, Braid provides a more accountable form of anonymity, and that the public benefit of a more Sybil-resistant Web3 outweighs the risks.

Open Science

In accordance with the ACM CCS open-science policy, we have made all artifacts necessary to reproduce and evaluate the contributions of this paper publicly available. Our goal is to facilitate rigorous review and enable future research to build upon our work.

Artifacts provided. The following artifacts are available:

- *Source code*: The complete source code for the Braid protocol, including the zero-knowledge proof circuits written using the gnark library, off-chain helper scripts, and the on-chain smart contracts written in Solidity.
- *Evaluation*: All scripts used to conduct performance benchmarks presented in Section 5, including micro-benchmarks, end-to-end latency tests, and throughput analysis.

Access to artifacts. All artifacts are hosted in a single, anonymized Git repository to facilitate the double-blind review process. The repository can be accessed at the following URL: https://anonymous.4open.science/r/Braid-D3D4.

A Full Braid protocol and extensions

This section systematically presents the complete Braid protocol, including its extensions that provide additional features and security guarantees. Figure 5 formalizes the main protocol, while Figures 12 and 14 provide formal definitions for association updates and credential management.

A.1 Identifier accountability

Braid allows $\mathcal{V}$ to request $\mathcal{R}$ to block the aid when detecting malicious behavior by $\mathcal{H}$. This mechanism enhances accountability compared to previous schemes by making all identifiers linked to aid unusable. However, this process requires the verifier to additionally validate the legitimacy of aid using the nullifier during credential verification, introducing *nullifier-based correlations*. To address this issue, Braid enables $\mathcal{H}$ to proactively refresh its aid, as shown in Figure 12.

Identifier blocklisting. As discussed in Section 4.2, whenever $\mathcal{H}$ presents cred, it must submit a nullifier n_a to $\mathcal{V}$. This n_a is structurally similar to aid but uses a different hash function. If $\mathcal{H}$ is suspected of significant malicious behavior (e.g., money laundering) $\mathcal{V}$ can forward n_a to $\mathcal{R}$, along with detailed evidence about $\mathcal{H}$'s violations. If $\mathcal{R}$ determines that $\mathcal{H}$'s actions warrant complete blocking, it records n_a to invalidate aid. This invalidation prevents $\mathcal{H}$ from successfully authenticating any future presentations of aid or credentials cred linked to its component identifiers id_i.

Association refreshing. To mitigate nullifier-based correlation, Braid allows $\mathcal{H}$ to autonomously refresh the associated identifier aid while preserving its components. Recall that we introduce a nonce u_a in aid that $\mathcal{H}$ can increment to refresh the association. When $\mathcal{H}$ detects correlation risks from using the same n_a across recent credential presentations, it can break association-level linkage by incrementing the nonce u_a. Concretely, $\mathcal{H}$ increments the nonce u_a, and proves in zero-knowledge that:

❶ aid derives from $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with the nonce u_a,
❷ aid′ derives from $\mathsf{id}_1, \ldots, \mathsf{id}_\ell$ with another nonce u'_a,
❸ aid is recorded by the ledger and not *blocklisted*, and
❹ The difference between u_a and u'_a is 1.

This emits a proof π_d for relation R_d:

$$\mathsf{aid} = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a) \ \wedge \ \mathsf{aid}' = H_a(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u'_a)$$
$$\wedge \ \mathsf{Auth}(r_a, \mathsf{aid}, \rho_a) = 1 \ \wedge \ n_a = H_n(\mathsf{id}_1, \ldots, \mathsf{id}_\ell, u_a)$$
$$\wedge \ u'_a = u_a + 1.$$

The proof π_d demonstrates that both pre- and post-update aid values derive from the same set of identifiers. Finally, $\mathcal{H}$ submits n_a to $\mathcal{R}$ to invalidate the previous version of aid.

As illustrated in Figure 13, this refreshing mechanism creates an *implicit chain* throughout each aid's lifecycle, with zero-knowledge proofs ensuring the refreshing validity. During updates, $\mathcal{R}$ discards the outdated n_a and generates a new aid, ensuring only the latest aid remains valid. When aid is *blocklisted*, this chain terminates, preventing further updates since the latest n_a required for updating has already been *invalidated*. Moreover, since there is no cryptographic link between the aid before and after refreshing, curious parties cannot track aid by monitoring VDR $\mathcal{R}$ records.

A.2 Credential revocation

Existing schemes commonly provide credential revocation through revocation lists with non-membership proofs [16, 23, 48, 60, 61]. During presentations, holders must prove their credentials do not appear on these lists. However, when credentials are selectively disclosed, verifiers see only partial credential information, preventing them from identifying specific credentials or connecting them to malicious activity. In contrast, Braid allows verifiers to report misconduct to issuers without full knowledge of credential contents or holder identities. Braid also enables holders to proactively request revocation of their own credentials, as defined in Figure 14.

Credential issuance. Braid's *credential issuance* is compatible with legacy systems, enabling a smooth transition without requiring modifications to existing credentials. To track the validity of issued credentials, each $\mathcal{I}$ maintains a smart contract containing a Merkle tree T_c. When $\mathcal{I}$ issues a credential cred to $\mathcal{H}$, the latter samples a random nonce u^c and commits to the credential cred by $c^c := H_a(\mathsf{cred}, u^c)$. $\mathcal{H}$ then generates a proof π_t to justify the commitment relation and sends it to $\mathcal{I}$. After verifying π_t, $\mathcal{I}$ inserts c^c into T_c, confirming that cred is registered in $\mathcal{I}$'s contract.

Credential revocation. Braid enables verifiers to initiate credential revocation. When $\mathcal{I}$ receives a document doc from a $\mathcal{V}$ describing credential misuse, it can invalidate the accused credential cred by recording its n^c value to its on-chain smart contract. Because this revocation mechanism is fully decentralized and relies exclusively on append-only on-chain state updates, the issuer $\mathcal{I}$ does not need to be online or actively participate during any credential presentation or verification process. Braid also empowers holders to voluntarily request revocation by submitting a doc with proof of ownership for cred.

Credential refreshing. This revocation mechanism also suffers from *nullifier-based correlation*, as the provided n^c remains constant when $\mathcal{H}$ presents the same cred multiple times. To address this issue, Braid allows $\mathcal{H}$ to proactively update the nullifier n^c and the commitment c^c by modifying the nonce u^c. Specifically, $\mathcal{H}$ samples a new randomness $u^{c\prime}$ and derives a new $c^{c\prime}$ from it. Then, $\mathcal{H}$ proves in zero-knowledge that:

❶ c^c is recorded by $\mathcal{I}$,
❷ n^c is not `revoked` in the issuer's revocation list, and
❸ $c^{c\prime}$ derives from cred with another nonce $u^{c\prime}$.

This emits a proof π_u for relation R_u:

$$\mathsf{Auth}(r_c, c^c, \rho_c) = 1 \ \wedge \ n^c = H_n(\mathsf{cred}, u^c)$$
$$\wedge \ c^c = H_a(\mathsf{cred}, u^c) \ \wedge \ c^{c\prime} = H_a(\mathsf{cred}, u^{c\prime}).$$

Credential management. Figure 14 describes the protocol for credential management and revocation. To support these functions, each issuer maintains an incremental Merkle tree T_c with credential commitments c^c as leaf nodes. The issuer also maintains a nullifier list N_c to track revoked credentials through their n^c markers.

A.3 Impersonation resistance

A significant challenge when using zero-knowledge proofs for selective credential disclosure is that verifiers may misuse these proofs. When $\mathcal{H}$ presents a proof π_c for credential cred to $\mathcal{V}$, a malicious $\mathcal{V}$ could impersonate $\mathcal{H}$ by forwarding π_c to another verifier $\mathcal{V}'$. To

Protocol Π_{BRAID}, extension 1

This protocol executes between a $\mathcal{P}$ and the data registry $\mathcal{R}$, where $\mathcal{P}$ could be any possible entity other than $\mathcal{R}$.

Association Appending: On receiving $(\texttt{append}, \text{sid}, \text{aid}, \text{id}_{\ell+1})$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ retrieves $(\text{aid}, \text{id}_1, \ldots, \text{id}_\ell, u_a)$ and $(\text{aid}, r_a, \rho_a)$ from its record, along with $(\text{id}_i, \text{pk}_i, \text{sk}_i)$ and $(\text{id}_i, h_i, r_i, \rho_i)$ for all $i \in [\ell + 1]$. Then, $\mathcal{P}$ computes $\text{aid}' := H_a(\text{id}_1, \ldots, \text{id}_{\ell+1}, u_a)$, $n_a := H_n(\text{id}_1, \ldots, \text{id}_\ell, u_a)$, and $n_{\ell+1} := H_n(\text{id}_{\ell+1}, \text{sk}_{\ell+1})$. Finally, $\mathcal{P}$ generates a proof π_p for relation R_p, and sends $(r_a, r_{\ell+1}, n_a, n_{\ell+1}, u_a, \text{aid}', \pi_p)$ to $\mathcal{R}$.

(2) $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, r_a)$, $(\texttt{check}, \text{sid}, r_{\ell+1})$, $(\texttt{check}, \text{sid}, n_a)$ and $(\texttt{check}, \text{sid}, n_{\ell+1})$ to $\mathcal{L}$. On receiving checked for $r_a, r_{\ell+1}$ and unrecorded for $n_a, n_{\ell+1}$, $\mathcal{R}$ verifies π_p and aborts if fails. Else, $\mathcal{R}$ sends $(\texttt{record}, \text{sid}, \text{aid}')$, $(\texttt{invalidate}, \text{sid}, n_a)$ and $(\texttt{invalidate}, \text{sid}, n_{\ell+1})$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r'_a, \rho'_a)$ and $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{R}$ sends (r'_a, ρ'_a) to $\mathcal{P}$.

(3) $\mathcal{P}$ records $(\text{aid}', r'_a, \rho'_a)$ and $(\text{aid}', \text{id}_1, \ldots, \text{id}_{\ell+1}, u_a)$, and then outputs $(\texttt{appended}, \text{sid}, \text{aid}')$.

Association Merging: On receiving $(\texttt{merge}, \text{sid}, \text{aid}_1, \text{aid}_2)$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ retrieves $(\text{aid}_1, \text{id}_1, \ldots, \text{id}_t, u_{a1})$, $(\text{aid}_2, \text{id}_{t+1}, \ldots, \text{id}_\ell, u_{a2})$, $(\text{aid}_1, r_{a1}, \rho_{a1})$ and $(\text{aid}_2, r_{a2}, \rho_{a2})$ from its record. Then, $\mathcal{P}$ computes $n_{a1} := H_n(\text{id}_1, \ldots, \text{id}_t, u_{a1})$, $n_{a2} := H_n(\text{id}_{t+1}, \ldots, \text{id}_\ell, u_{a2})$ and $\text{aid}' := H_a(\text{id}_1, \ldots, \text{id}_\ell, u'_a)$, where $u'_a := \max(u_{a1}, u_{a2})$. Finally, $\mathcal{P}$ generates a proof π_m for relation R_m, and sends $(r_{a1}, r_{a2}, n_{a1}, n_{a2}, u_{a1}, u_{a2}, \text{aid}', \pi_m)$ to $\mathcal{R}$.

(2) $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, r_{a1})$, $(\texttt{check}, \text{sid}, r_{a2})$, $(\texttt{check}, \text{sid}, n_{a1})$ and $(\texttt{check}, \text{sid}, n_{a2})$ to $\mathcal{L}$. On receiving checked for r_{a1}, r_{a2} and unrecorded for n_{a1}, n_{a2}, $\mathcal{R}$ verifies π_m and aborts if fails. Else, $\mathcal{R}$ sends $(\texttt{record}, \text{sid}, \text{aid}')$, $(\texttt{invalidate}, \text{sid}, n_{a1})$ and $(\texttt{invalidate}, \text{sid}, n_{a2})$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r'_a, \rho'_a)$ and $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{R}$ sends (r'_a, ρ'_a) to $\mathcal{P}$.

(3) $\mathcal{P}$ records $(\text{aid}', r'_a, \rho'_a)$ and $(\text{aid}', \text{id}_1, \ldots, \text{id}_\ell, u'_a)$, and then outputs $(\texttt{merged}, \text{sid}, \text{aid}')$.

Association Refreshing: On receiving $(\texttt{refresh}, \text{sid}, \text{aid})$, party $\mathcal{P}$ performs as follows:

(1) $\mathcal{P}$ retrieves $(\text{aid}, \text{id}_1, \ldots, \text{id}_\ell, u_a)$ and $(\text{aid}, r_a, \rho_a)$ from its record. Then, $\mathcal{P}$ computes $n_a := H_n(\text{id}_1, \ldots, \text{id}_\ell, u_a)$ and $\text{aid}' := H_a(\text{id}_1, \ldots, \text{id}_\ell, u'_a)$ where $u'_a := u_a + 1$. Finally, $\mathcal{P}$ generates a proof π_d for relation R_d, and sends $(r_a, u_a, n_a, \text{aid}', \pi_d)$ to $\mathcal{R}$.

(2) $\mathcal{R}$ sends $(\texttt{check}, \text{sid}, r_a)$ and $(\texttt{check}, \text{sid}, n_a)$ to $\mathcal{L}$. On receiving checked for r_a and unrecorded for n_a, $\mathcal{R}$ verifies π_d and aborts if fails. Else, $\mathcal{R}$ sends $(\texttt{record}, \text{sid}, \text{aid}')$ and $(\texttt{invalidate}, \text{sid}, n_a)$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r'_a, \rho'_a)$ and $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{R}$ sends (r'_a, ρ'_a) to $\mathcal{P}$.

(3) $\mathcal{P}$ records $(\text{aid}', r'_a, \rho'_a)$ and $(\text{aid}', \text{id}_1, \ldots, \text{id}_\ell, u'_a)$, and then outputs $(\texttt{refreshed}, \text{sid}, \text{aid}')$.

Figure 12: BRAID Protocol extension: association update.

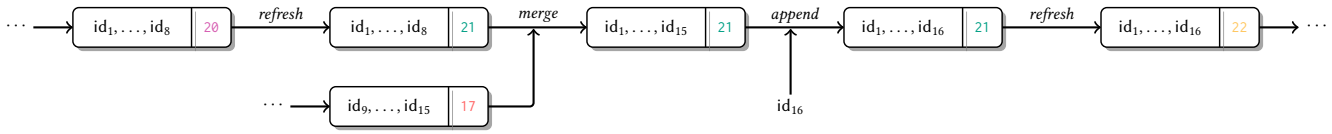

Figure 13: The lifecycle of an associated identifier, with pointers indicating updates (e.g., merging, appending, and refreshing) whose correctness is guaranteed by cryptographic proofs. Only the latest state is considered valid.

Protocol Π_{BRAID}, extension 2

This protocol executes between the issuer $\mathcal{I}$, the holder $\mathcal{H}$, and the verifier $\mathcal{V}$. All participants have access to the data registry $\mathcal{R}$ and the decentralized ledger $\mathcal{L}$. The operations of $\mathcal{I}$ in the revocation and refreshing phases are executed autonomously by its on-chain smart contract, which interacts with $\mathcal{L}$ to manage credential states.

Credential Issuance: On receiving $(\texttt{issue}, \text{sid}, \text{id}^{\mathcal{H}}, s)$, issuer $\mathcal{I}$ performs as follows:

(1) $\mathcal{I}$ retrieves its keypair $(\text{sk}^{\mathcal{I}}, \text{pk}^{\mathcal{I}})$, signs the claim s by $\sigma := \text{Sign}(\text{sk}^{\mathcal{I}}, (\text{id}^{\mathcal{I}}, \text{id}^{\mathcal{H}}, s))$, and forms the credential by $\text{cred} := (\text{id}^{\mathcal{I}}, \sigma, \text{id}^{\mathcal{H}}, s)$. $\mathcal{I}$ sends cred to $\mathcal{H}$.

(2) $\mathcal{H}$ samples $u^c \leftarrow_{\$} \mathbb{Z}_p$ and computes $c^c := H_a(\text{cred}, u^c)$. Then, $\mathcal{H}$ generates a proof π_t for relation R_t, and sends (c^c, π_t) to $\mathcal{I}$.

(3) $\mathcal{I}$ verifies π_t and aborts if fails. Else, $\mathcal{I}$ sends $(\texttt{record}, \text{sid}, c^c)$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r_c, \rho_c)$ from $\mathcal{L}$, $\mathcal{I}$ returns (r_c, ρ_c) to $\mathcal{H}$.

(4) $\mathcal{H}$ records $(\text{cred}, u^c, c^c, r_c, \rho_c)$ and outputs $(\texttt{issued}, \text{sid})$.

Credential Revocation: On receiving $(\texttt{revoke}, \text{sid}, n^c, \text{doc})$, a party $\mathcal{P}$ (can be either $\mathcal{H}$ or $\mathcal{V}$) performs as follows:

(1) $\mathcal{P}$ sends (n^c, doc) to $\mathcal{I}$.

(2) $\mathcal{I}$ checks doc to confirm that cred should be revoked. If this is the case, $\mathcal{I}$ sends $(\texttt{invalidate}, \text{sid}, n^c)$ to $\mathcal{L}$. On receiving $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{I}$ outputs $(\texttt{revoked}, \text{sid})$.

Credential Refreshing: On receiving $(\texttt{refresh}, \text{sid}, \text{cred})$, $\mathcal{H}$ performs as follows:

(1) $\mathcal{H}$ retrieves $(\text{cred}, u^c, c^c, r_c, \rho_c)$ from its record. Then, $\mathcal{H}$ samples $u^{c\prime} \leftarrow_{\$} \mathbb{Z}_p$, and computes $c^{c\prime} := H_a(\text{cred}, u^{c\prime})$ and $n^c := H_n(\text{cred}, u^c)$. $\mathcal{H}$ generates a proof π_u for relation R_u, and sends $(r_c, c^{c\prime}, n^c, \pi_u)$ to $\mathcal{I}$.

(2) $\mathcal{I}$ sends $(\texttt{check}, \text{sid}, r_c)$ and $(\texttt{check}, \text{sid}, n^c)$ to $\mathcal{L}$. On receiving checked for r_c and unrecorded for n^c, $\mathcal{I}$ verifies π_u and aborts if fails. Then, $\mathcal{I}$ sends $(\texttt{record}, \text{sid}, c^{c\prime})$ and $(\texttt{invalidate}, \text{sid}, n^c)$ to $\mathcal{L}$. On receiving $(\texttt{recorded}, \text{sid}, r'_c, \rho'_c)$ and $(\texttt{invalidated}, \text{sid})$ from $\mathcal{L}$, $\mathcal{I}$ returns (r'_c, ρ'_c) to $\mathcal{H}$.

(3) $\mathcal{H}$ records $(\text{cred}, u^{c\prime}, c^{c\prime}, r'_c, \rho'_c)$ and outputs $(\texttt{refreshed}, \text{sid})$.

Figure 14: BRAID Protocol extension: credential management and revocation.

prevent such impersonation attacks, BRAID implements two mechanisms allowing $\mathcal{H}$ to specify a designated $\mathcal{V}$, thereby preventing the presentation process from being replayed to other $\mathcal{V}'$.

Interactive verification. One solution requires π_c to contain a component specified by $\mathcal{V}$. Here, $\mathcal{V}$ samples a challenge e and sends it to $\mathcal{H}$ before cred is presented. $\mathcal{H}$ must incorporate e into R_c for binding purposes. If $\mathcal{V}$ later attempts to pass π_c to another verifier $\mathcal{V}'$ in a different presentation, $\mathcal{V}'$ would need to provide an e' that matches the original challenge e exactly—an occurrence with negligible probability.

Key attestation. To avoid this additional round of interaction, BRAID implements an alternative mechanism by allowing $\mathcal{H}$ to embed elements that only $\mathcal{V}$ knows (i.e., $\mathcal{V}$'s private key $\text{sk}^{\mathcal{V}}$) into

relations [65]. The credential presentation then becomes a proof of knowledge with two possible conditions:

- Either relation R_c is satisfied, or
- $\mathcal{V}$ knows a $\mathsf{sk}^{\mathcal{V}}$ such that $\mathsf{pk}^{\mathcal{V}} = \mathsf{PKGen}(\mathsf{sk}^{\mathcal{V}})$.

$\mathcal{V}$ will accept this proof since it knows it didn't present the credential itself, meaning R_c must be satisfied. However, when $\mathcal{V}$ forwards this proof to another $\mathcal{V}'$, the latter will not accept R_c since $\mathcal{V}$ could have created a valid proof simply by using its private key $\mathsf{sk}^{\mathcal{V}}$.

A.4 Immediate verification

To prevent $\mathcal{H}$ from bypassing the anti-Sybil checks of $\mathcal{V}$ by updating aid during a campaign, we introduced a countermeasure in Section 4.2. This involves delaying all verifications until the end of the campaign interaction period, i.e., after all users have provided their proofs. However, some scenarios require immediate verification and completion of interactions when users submit their credentials.

We provide a solution for this scenario here. The core idea is to require $\mathcal{H}$ to have generated the aid before the start of the campaign[5]. For this, $\mathcal{V}$ requires that the proof π_c provided by $\mathcal{H}$, in addition to satisfying the relation R_c, must also include a constraint that the index of aid is no greater than a threshold t, which is the number of elements in the Merkle tree T of $\mathcal{R}$ at the start of campaign eid initiated by $\mathcal{V}$, that is

$$\mathsf{index}(\mathsf{aid}) < t = |T|.$$

Performance optimization. When the number of elements in T is large, the range proof for index(aid) will significantly slow down the generation time of π_c, since the arithmetic circuit needs to first convert index(aid) and t into binary representations and then compare them bit by bit. However, it is not necessary to perform this conversion in the circuit. An optimization is to take advantage of the Merkle proof ρ_a with respect to aid. It turns out that ρ_a contains a binary index array $\boldsymbol{d}$, which records the relative position of the two elements in each hash on the Merkle path. The reverse array $\mathsf{rev}(\boldsymbol{d})$ is exactly the binary representation of index(aid). Therefore, the above constraint can be transformed into $\mathsf{rev}(\boldsymbol{d}) < \mathsf{bin}(t)$, where $\mathsf{bin}(t)$ refers to the binary representation of t, which can be pre-calculated by $\mathcal{V}$ and encoded into the circuit. This constraint compares the binary values in the two arrays sequentially with a complexity no greater than the number of layers in T.

B Formal definitions

B.1 Non-interactive zero knowledge

Definition 2 (zkSNARK). We call Π *a zero-knowledge succinct non-interactive argument of knowledge (**zkSNARK**) for* $\mathcal{RG}$ *if it has **zero-knowledge** (Definition 3), **succinctness** (Definition 4), **completeness** (Definition 5) and **computational knowledge soundness** (Definition 6).*

Definition 3 (Zero-Knowledge). Conceptually, an argument is zero-knowledge if it does not give away any information other than the truth of the statement. We say Π *is **zero-knowledge** if for all* $\lambda \in \mathbb{N}$, $R \in \mathcal{RG}_\lambda$, $(x, w) \in R$ *and all adversaries* $\mathcal{A}$, *there exists a simulator* Sim *such that,*

$$\Pr\left[\begin{matrix}\varsigma \leftarrow_\$ \mathsf{Setup}(R) \\ \pi \leftarrow_\$ \mathsf{Prove}(\varsigma, x, w)\end{matrix} : \mathcal{A}(R, \varsigma, \pi) = 1\right] = \Pr\left[\begin{matrix}\varsigma \leftarrow_\$ \mathsf{Setup}(R) \\ \pi \leftarrow_\$ \mathsf{Sim}(R, x)\end{matrix} : \mathcal{A}(R, \varsigma, \pi) = 1\right].$$

Definition 4 (Succinctness). We say Π *is **succinct** if the verifier runtime can be bounded by a polynomial in* $\lambda + |x|$ *and the proof size can be bounded by a polynomial in* λ. *On this basis, we claim that* Π *is **completely succinct** if the length of the reference string* $|\varsigma|$ *can also be bounded by a polynomial in* λ.

Definition 5 (Completeness). Intuitively, completeness means that given any true statement, an honest prover is necessarily able to convince an honest verifier to accept the argument. We say Π *is **complete** if for all* $\lambda \in \mathbb{N}$, $R \in \mathcal{RG}_\lambda$ *and* $(x, w) \in R$,

$$\Pr\left[\begin{matrix}\varsigma \leftarrow_\$ \mathsf{Setup}(R) \\ \pi \leftarrow_\$ \mathsf{Prove}(\varsigma, x, w)\end{matrix} : \Pi.\mathsf{Verify}(\varsigma, x, \pi) = 1\right] = 1.$$

Definition 6 (Soundness). We say Π *is **computational knowledge sound** if for all non-uniform polynomial-time adversaries* $\mathcal{A}$, *there exists a non-uniform polynomial-time extractor* Ext *such that,*

$$\Pr\left[\begin{matrix}\varsigma \leftarrow_\$ \mathsf{Setup}(R) \\ (x, \pi) \leftarrow_\$ \mathcal{A} : \\ w \leftarrow \mathsf{Ext}(\varsigma, x, \pi)\end{matrix}\begin{matrix}\\ (x, w) \notin R \ \wedge \\ \mathsf{Verify}(\varsigma, x, \pi) = 1\end{matrix}\right] \leq \mathsf{negl}(\lambda).$$

B.2 Commitment scheme

Definition 7 (Commitment). We call Γ *a secure cryptographic commitment scheme if it has the property of **correctness** defined above, along with the properties of **hiding** (Definition 8) and **binding** (Definition 9).*

Definition 8 (Hiding). Intuitively, hiding means that the commitment c should reveal nothing about the committed message m. We say Γ *is **hiding** if for all efficient adversaries* $\mathcal{A}$, *two different messages* m_0, m_1 *and two arbitrary opening strings* u_0, u_1,

$$\left|\Pr\left[\begin{matrix}c_0 \leftarrow_\$ \mathsf{Commit}(m_0, u_0) \\ r_0 \leftarrow \mathcal{A}(c_0)\end{matrix} : r_0 = 0\right] - \Pr\left[\begin{matrix}c_1 \leftarrow_\$ \mathsf{Commit}(m_1, u_1) \\ r_1 \leftarrow \mathcal{A}(c_1)\end{matrix} : r_1 = 0\right]\right| \leq \mathsf{negl}(\lambda)$$

holds.

Definition 9 (Binding). Binding indicates that a commitment c could only open to a single message m. We say Γ *is **binding** if for all efficient adversaries* $\mathcal{A}$ *which outputs* (c, m_0, m_1, u_0, u_1),

$$\Pr\left[\begin{matrix}m_0 \neq m_1 \\ \Gamma.\mathsf{Open}(m_0, c, u_0) = \Gamma.\mathsf{Open}(m_1, c, u_1)\end{matrix}\right] \leq \mathsf{negl}(\lambda)$$

holds.

C Sybil-resistance proof

To prove Theorem 1, we first consider the holder $\mathcal{H}$'s credential selection strategy. At the q-th campaign, $\mathcal{H}$ must select k credentials from a pool of $s(q)$ credentials satisfying the predicate specified by the verifier. The association rules constrain $\mathcal{H}$'s choices as follows:

[5] To afford potential participants ample time to prepare associated identifiers, $\mathcal{V}$ may pre-announce the commencement time of the campaign.

- If no association operations occur, $\mathcal{H}$ selects k identifiers freely. The number of possible combinations is $\binom{s(q)}{k}$.
- If $\mathcal{H}$ reuses a set of identifiers S_j in an association, then the selection space collapses to the subsets of this association set S_j. By the association rule, this does not increase $s(q)$.
- If $\mathcal{H}$ triggers overlap associations, i.e., $\mathcal{H}$ chooses identifiers from the intersection of two or more association sets, or identifiers from an association set, along with some identifiers not yet associated, the resulting association set $S_{\text{associated}}$ permanently links all overlapping identifiers. Subsequent selections involving any identifiers in $S_{\text{associated}}$ must include the entire set.

Forcing associations reduces $\mathcal{H}$'s future flexibility. After t associations, $\mathcal{H}$'s effective selection space is upper bounded by the original unassociated selection space:

$$\binom{s(q)}{k} \geq \sum_{t=0}^{k} \binom{\#\text{ associated clusters}}{t} \cdot \binom{s(q) - t \cdot \ell_{\text{avg}}}{k-t},$$

where ℓ_{avg} is the average size of associated clusters. The number of associated clusters is at most k, and the number of identifiers in each cluster is at most ℓ_{avg}. The total number of identifiers in the selection space is $s(q) - t \cdot \ell_{\text{avg}}$. Thus, $\binom{s(q)}{k}$ dominates all constrained selection scenarios.

Next, we consider the probability that $\mathcal{H}$ can avoid association when attempting an attack. For an attack to succeed, the k credentials presented must not be bound to the same associated identifier. The probability when presenting k credentials simultaneously is:

$$\left(\frac{1}{\ell(q)^{k-1}}\right) \cdot \left(\frac{\ell(q)-1}{\ell(q)^{k}}\right) = \frac{\ell(q)-1}{\ell(q)^{2k-1}}.$$

Then, we consider the hash consistency. Note that each association operation generates a hash $\mathsf{aid} = H_a(S)$ for an identifier set S. $\mathcal{H}$ can only launch a Sybil attack if:

- All hashes generated during the associations are consistent with previous commitments.
- No hash collisions occur between distinct association sets.

Let B be the event that $\mathcal{H}$ finds distinct identifier sets $S_1 \neq S_2$ such that $H_a(S_1) = H_a(S_2)$. By the collision resistance of the hash function H_a, the probability is bounded by:

$$\Pr[B] \leq \frac{T(q)^2}{2^\lambda},$$

where $T(q)$ is the total number of association operations. Since $T(q) \leq q \cdot \ell(q)$ (at most one association operation per identifier per campaign), we have:

$$\Pr[B] \leq \frac{T(q)^2}{2^\lambda} \leq \frac{q^2 \cdot \ell(q)^2}{2^\lambda}.$$

Finally, we bound the success probability of $\mathcal{H}$'s Sybil attack. By taking the union bound over the probability of finding a valid unassociated credential combination and the probability of a hash collision, we have:

$$\Pr[W] \leq \binom{s(q)}{k}\left(\frac{\ell(q)-1}{\ell(q)^{2k-1}}\right) + \frac{q^2 \cdot \ell(q)^2}{2^\lambda}.$$

The term $(q \cdot \ell(q))^2/2^\lambda$ is negligible in λ due to $|\mathbb{Z}_p| \geq 2^\lambda$ and $\ell(q) = \mathsf{poly}(\lambda)$. Thus, we have:

$$\Pr[W] \leq \binom{s(q)}{k}\left(\frac{\ell(q)-1}{\ell(q)^{2k-1}}\right) + \mathsf{negl}(\lambda).$$

D Security proof

To prove Theorem 2, we demonstrate that no p.p.t. environment $\mathcal{E}$ can differentiate between the ideal process of functionality $\mathcal{F}_{\text{BRAID}}$ and the execution of protocol Π_{BRAID} in the $\mathcal{L}$-hybrid model.

Concretely, parties in protocol Π_{BRAID} execute alongside an adversary $\mathcal{A}$ and $\mathcal{E}$ with initial input z. Within the $\mathcal{L}$-hybrid world, all parties are granted access to the ledger functionality $\mathcal{L}$. The adversary $\mathcal{A}$ can exchange messages with parties, read their outgoing messages, and deliver these messages to other parties. Upon corrupting a party, $\mathcal{A}$ gains access to all its internal states and controls its subsequent actions. Meanwhile, the ideal functionality $\mathcal{F}_{\text{BRAID}}$ executes with an ideal world adversary (simulator) $\mathcal{S}$ and $\mathcal{E}$ via a set of *dummy parties*. These dummy parties relay messages between $\mathcal{E}$ and $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ can exchange backdoors with $\mathcal{F}_{\text{BRAID}}$ and is tasked with delivering $\mathcal{F}_{\text{BRAID}}$'s outgoing messages to the dummy parties. When $\mathcal{S}$ corrupts a party, both $\mathcal{E}$ and $\mathcal{F}_{\text{BRAID}}$ are notified.

To demonstrate indistinguishability, we show that for any adversary $\mathcal{A}$, we can construct a simulator $\mathcal{S}$ such that the environment $\mathcal{E}$ cannot distinguish between the output distributions of the ideal world and the hybrid world executions. Since $\mathcal{R}$ is implemented as an on-chain contract with public and immutable internal states, we need only consider the following cases:

- All parties are uncorrupted,
- The holder $\mathcal{H}$ is corrupted,
- The issuer $\mathcal{I}$ and the verifier $\mathcal{V}$ are corrupted, or
- All parties except $\mathcal{R}$ are corrupted.

Here, we give a proof sketch containing the construction of the simulator $\mathcal{S}$ for all corruption cases and the corresponding description.

All uncorrupted. In this case, it suffices for the simulator $\mathcal{S}$ to generate the transcript of all messages of the execution of Π_{BRAID} towards the adversary $\mathcal{A}$ and thus $\mathcal{E}$.

(1) If an uncorrupted party $\mathcal{P}$ (either $\mathcal{I}$ or $\mathcal{H}$) is activated by $(\texttt{register}, \mathsf{sid})$, $\mathcal{S}$ obtains id from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ samples $\mathsf{sk}^* \leftarrow_\$ \mathbb{Z}_p$, computes $\mathsf{pk}^* = \mathsf{PKGen}(\mathsf{sk}^*)$ and $h^* = H_a(\mathsf{id}, \mathsf{sk}^*)$. Then, $\mathcal{S}$ sends $(\texttt{register}, \mathsf{sid}, \mathsf{id}, \mathsf{pk}^*)$ and $(\texttt{record}, \mathsf{sid}, h^*)$ to $\mathcal{L}$, and records the returned $(\mathsf{id}, \mathsf{pk}^*, \mathsf{sk}^*)$ and $(\mathsf{id}, h^*, r^*, \rho^*)$.

(2) If an uncorrupted party $\mathcal{P}$ is activated by $(\texttt{associate}, \mathsf{sid}, \{\mathsf{id}_i\}_\ell)$, $\mathcal{S}$ obtains aid from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ retrieves $\mathsf{sk}_i^* \leftarrow_\$ \mathbb{Z}_p$ for all $i \in [\ell]$, and computes $n_i^* := H_n(\mathsf{id}_i, \mathsf{sk}_i^*)$. Finally, $\mathcal{S}$ sends $(\texttt{record}, \mathsf{sid}, \mathsf{aid})$ and $(\texttt{invalidate}, \mathsf{sid}, n_i^*)$ to $\mathcal{L}$, and records the returned $(\mathsf{aid}, r_a, \rho_a)$, along with $(\mathsf{aid}, \mathsf{id}_1, \ldots, \mathsf{id}_\ell, 0)$.

(3) If an uncorrupted party $\mathcal{P}$ is activated by an association update (i.e., `append`, `merge`, or `refresh`), $\mathcal{S}$ obtains the updated aid' from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ retrieves dummy nonces and keys as needed to compute dummy nullifiers n^*. $\mathcal{S}$ generates the simulated zero-knowledge proof (π_p^*, π_m^*, or π_d^*) without the private witness (relying on the zero-knowledge property). Finally, $\mathcal{S}$ simulates the ledger interactions by sending

(record, sid, aid′) and the corresponding (invalidate, sid, n^*) messages to $\mathcal{L}$, recording the returned tree roots and nullifiers.

(4) If an uncorrupted holder $\mathcal{H}$ is activated by (present, sid, aid, $\{\mathsf{cred}_i\}_n$), $\mathcal{S}$ obtains $\{\mathsf{id}_i^{\mathcal{I}}\}_n$ from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ sends (retrieve, sid, $\mathsf{id}_i^{\mathcal{I}}$) to $\mathcal{L}$, receiving $\mathsf{pk}_i^{\mathcal{I}}$ for all $i \in [n]$. Then, $\mathcal{S}$ generates dummy public nullifiers and constructs the simulated proof π_c^* without the witnesses to include in the transcript. Finally, $\mathcal{S}$ simulates $\mathcal{V}$'s verification by sending the corresponding (check, sid, ·) requests to $\mathcal{L}$ for the roots and nullifiers.

(5) If an uncorrupted party $\mathcal{P}$ is activated by (recover, sid, aid, id, $\{\mathsf{id}'\}_t$, $\{\mathsf{sk}'\}_t$), $\mathcal{S}$ constructs the simulated proof π_k^* and sends it to $\mathcal{R}$ in the transcript. Then, $\mathcal{S}$ samples $\mathsf{sk}^* \leftarrow_\$ \mathbb{Z}_p$, computes $\mathsf{pk}^* = \mathsf{PKGen}(\mathsf{sk}^*)$, $h^* = H_a(\mathsf{id}, \mathsf{sk}^*)$ and $n^* = H_n(\mathsf{id}, \mathsf{sk}^*)$. Finally, $\mathcal{S}$ sends (register, sid, id, pk^*), (record, sid, h^*) and (invalidate, sid, n^*) to $\mathcal{L}$, and records the returned (id, pk^*, sk^*) and (id, h^*, r^*, ρ^*).

Execution of $\mathcal{F}_{\text{BRAID}}$ in the ideal world is indistinguishable from the execution of Π_{BRAID} in the $\mathcal{L}$-hybrid world given the collision-resistance of H_a, H_n and the hash primitive of the Merkle tree.

Corrupted holder. In this case, the simulator $\mathcal{S}$ needs to generate the transcript of all messages of the execution of Π_{BRAID}, along with the outputs of the corrupted holder $\mathcal{H}$ towards $\mathcal{F}_{\text{BRAID}}$ and $\mathcal{E}$. This indicates that whenever $\mathcal{E}$ activates $\mathcal{H}$, its message will be forwarded to $\mathcal{S}$.

(1) If a corrupted holder $\mathcal{H}$ is activated by (register, sid), $\mathcal{S}$ learns (id, sk, pk) from $\mathcal{H}$ and check if $\mathsf{pk} = \mathsf{PKGen}(\mathsf{sk})$. If this is the case, $\mathcal{S}$ computes $h := H_a(\mathsf{id}, \mathsf{sk})$ and sends (register, sid, id, pk) and (record, sid, h) to $\mathcal{L}$. The simulator also forwards the extracted registration state to $\mathcal{F}_{\text{BRAID}}$, so that the ideal functionality records $(\mathcal{H}, \mathsf{id}, \mathsf{pk}, \mathsf{sk})$ consistently with the ledger transcript.

(2) If a corrupted holder $\mathcal{H}$ is activated by (associate, sid, $\{\mathsf{id}_i\}_\ell$), $\mathcal{S}$ computes $n_i := H_n(\mathsf{id}_i, \mathsf{sk}_i)$, and sends (record, sid, aid) and (invalidate, sid, n_i) to $\mathcal{L}$.

(3) If a corrupted holder $\mathcal{H}$ initiates an association update, $\mathcal{S}$ intercepts the zero-knowledge proof (π_p, π_m, or π_d) sent by $\mathcal{A}$ to $\mathcal{R}$. By utilizing the knowledge extractor of the zkSNARK (Definition 6), $\mathcal{S}$ extracts the required witnesses (e.g., the new identifier $\mathsf{id}_{\ell+1}$ or nonces). $\mathcal{S}$ then sends the corresponding (append, ...), (merge, ...), or (refresh, ...) command to $\mathcal{F}_{\text{BRAID}}$ on behalf of $\mathcal{H}$ and simulates the $\mathcal{L}$ state changes.

(4) If a corrupted holder $\mathcal{H}$ participates in a campaign, $\mathcal{A}$ submits a zero-knowledge proof π_c to $\mathcal{V}$. $\mathcal{S}$ uses the knowledge extractor of the zkSNARK to extract the underlying aid and credentials $\{\mathsf{cred}_i\}_n$ from π_c. $\mathcal{S}$ then sends (present, sid, aid, $\{\mathsf{cred}_i\}_n$) to $\mathcal{F}_{\text{BRAID}}$ on behalf of the corrupted $\mathcal{H}$ to synchronize the ideal state.

(5) If a corrupted holder $\mathcal{H}$ initiates a key recovery, $\mathcal{S}$ uses the knowledge extractor on the proof π_k (or π_k') to extract the target id, the threshold identifiers $\{\mathsf{id}'\}_t$, and their keys $\{\mathsf{sk}'\}_t$. $\mathcal{S}$ then sends (recover, sid, aid, id, $\{\mathsf{id}'\}_t$, $\{\mathsf{sk}'\}_t$) to $\mathcal{F}_{\text{BRAID}}$ on behalf of $\mathcal{H}$.

Corrupted issuer and verifier. Then, we consider the case when the issuer $\mathcal{I}$ and the verifier $\mathcal{V}$ are corrupted. In this case, the simulator $\mathcal{S}$ needs to generate the transcript of all messages in Π_{BRAID}, and the outputs of the corrupted $\mathcal{I}$ and $\mathcal{V}$.

(1) If a holder $\mathcal{H}$ is activated by (register, sid), $\mathcal{S}$ obtains id from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ samples $\mathsf{sk}^* \leftarrow_\$ \mathbb{Z}_p$, computes $\mathsf{pk}^* = \mathsf{PKGen}(\mathsf{sk}^*)$ and $h^* = H_a(\mathsf{id}, \mathsf{sk}^*)$. Then, $\mathcal{S}$ sends (register, sid, id, pk^*) and (record, sid, h^*) to $\mathcal{L}$, and records the returned (id, pk^*, sk^*) and (id, h^*, r^*, ρ^*).

(2) If a holder $\mathcal{H}$ is activated by (associate, sid, $\{\mathsf{id}_i\}_\ell$), $\mathcal{S}$ obtains aid from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ retrieves $\mathsf{sk}_i^* \leftarrow_\$ \mathbb{Z}_p$ for all $i \in [\ell]$, and computes $n_i^* := H_n(\mathsf{id}_i, \mathsf{sk}_i^*)$. Finally, $\mathcal{S}$ sends (record, sid, aid) and (invalidate, sid, n_i^*) to $\mathcal{L}$, and records the returned (aid, r_a, ρ_a), along with (aid, $\mathsf{id}_1, \ldots, \mathsf{id}_\ell, 0$).

(3) If an uncorrupted holder $\mathcal{H}$ is activated by an association update, $\mathcal{S}$ obtains the new aid′ from $\mathcal{F}_{\text{BRAID}}$. $\mathcal{S}$ generates the simulated proof (π_p^*, π_m^*, or π_d^*) without knowing the holder's private witness, and sends the corresponding ledger update requests to $\mathcal{L}$ in the name of the honest $\mathcal{H}$, recording the updated states.

(4) If a corrupted verifier $\mathcal{V}$ is activated by (campaign, sid, ϕ), $\mathcal{S}$ waits for a present message from $\mathcal{H}$. Since $\mathcal{H}$ is uncorrupted, $\mathcal{S}$ does not possess the private keys $\mathsf{sk}_i^{\mathcal{H}}$. Instead, $\mathcal{S}$ invokes the zero-knowledge simulator of the zkSNARK to generate a simulated proof π_c^* without the witness. $\mathcal{S}$ then sends this simulated proof to $\mathcal{V}$ on behalf of the honest $\mathcal{H}$, allowing the corrupted $\mathcal{V}$ to naturally query $\mathcal{L}$ in the hybrid world.

(5) If a holder $\mathcal{H}$ is activated by (recover, sid, aid, id, $\{\mathsf{id}'\}_t$, $\{\mathsf{sk}'\}_t$), $\mathcal{S}$ constructs the simulated proof π_k^* and sends it to $\mathcal{R}$ in the name of $\mathcal{A}$. Then, $\mathcal{S}$ samples $\mathsf{sk}^* \leftarrow_\$ \mathbb{Z}_p$, computes $\mathsf{pk}^* = \mathsf{PKGen}(\mathsf{sk}^*)$, $h^* = H_a(\mathsf{id}, \mathsf{sk}^*)$ and $n^* = H_n(\mathsf{id}, \mathsf{sk}^*)$. Finally, $\mathcal{S}$ sends (register, sid, id, pk^*), (record, sid, h^*) and (invalidate, sid, n^*) to $\mathcal{L}$, and records the returned (id, pk^*, sk^*) and (id, h^*, r^*, ρ^*).

Note that in this case, the behavior of the uncorrupted holder mirrors that of the first case.

All corrupted. In this case, the simulation of $\mathcal{S}$ is a mixture of the above two scenarios. $\mathcal{S}$ should simulate the interacting of $\mathcal{A}$ and $\mathcal{L}$, along with the messages between the parties.